

MSC ARTIFICIAL INTELLIGENCE
MASTER THESIS

Efficient Fine-Tuning of Diffusion Models with Differentiable Rewards

by
JAN HUTTER
14006979

June 19, 2026

36 EC
January 5 - June 19

Supervisor:

Luca Eyring, dr. Yingjun Du,
Prof. dr. Zeynep Akata, &
Prof. dr. Cees G. M. Snoek

Examiner:

Prof. dr. Paul Groth

Second reader:

Prof. dr. Cees G. M. Snoek



UNIVERSITEIT VAN AMSTERDAM

Contents

1	Introduction	1
1.1	Research Questions	2
1.2	Contributions	3
2	Background	4
2.1	Preliminaries	4
2.2	Diffusion Models	4
2.3	Continuous Normalizing Flows	5
2.4	Conditional Flow Matching	5
2.5	Inference and Sampling	6
2.6	GLASS Flows	7
2.7	Fine-Tuning Flow Matching Models & VLMs	8
2.8	Classifier-Free Guidance	8
3	Related Work	10
3.1	Gradient-based Optimization	10
3.2	RL-based Optimization	11
4	Gladjoint Matching	13
4.1	Regularization	14
4.2	Timestep Subsampling and Endpoint-Level Loss	14
5	Experiments	16
5.1	Experimental Settings	16
5.1.1	Evaluation Benchmarks	16
5.1.2	Implementation Details	17
5.2	Reward Hacking	17
5.2.1	Redness	17
5.2.2	NVILA	20
5.3	Sparse rewards	21
5.4	General Performance	22
6	Conclusion	27
	Appendix	35
A	Improved regularization	35
B	Hyperparameters	35
C	Redness ALL Hyperparameters	36
D	BikeReward prompt	38

Abstract

The alignment of continuous-time generative AI frameworks, such as diffusion and flow matching, with complex human preferences is a critical challenge in text-to-image generation. Current reward fine-tuning paradigms are generally divided into two distinct approaches: reinforcement learning methods, which bypass memory bottlenecks but suffer from slow convergence and poor credit assignment, and gradient-based optimization, which is highly efficient but notoriously susceptible to reward hacking and severe overfitting. We introduce Gladjoint Matching, a novel gradient-based fine-tuning method that uses GLASS transitions within Adjoint Matching training. This combination enables efficient reward optimization while preserving the stochasticity required for reward fine-tuning. Extensive experiments across synthetic environments and standard evaluation benchmarks, including GenEval and UniGenBench++, demonstrate that Gladjoint Matching mitigates the reward hacking prevalent in earlier gradient-based methods while converging significantly faster than current RL approaches. Furthermore, the proposed method proves highly robust under sparse reward conditions, ultimately surpassing RL-based baselines and offering a stronger robustness–efficiency trade-off than prior gradient-based methods.

Chapter 1

Introduction

Text-to-image generation has undergone a rapid evolution in recent years. Early successes in high-fidelity image synthesis were largely driven by Generative Adversarial Networks (GANs) (Goodfellow et al., 2014) and Variational Autoencoders (VAEs) (Kingma and Welling, 2022). GANs operate on an adversarial minimax objective, pitting a generator against a discriminator to produce exceptionally sharp and realistic samples. However, their adversarial nature renders training notoriously unstable (Arjovsky et al., 2017; Miyato et al., 2018). Conversely, VAEs provide a more mathematically principled probabilistic framework by maximizing the Evidence Lower Bound (ELBO) of the data likelihood. While VAEs guarantee stable optimization and yield highly structured, interpretable latent spaces, their reliance on simple surrogate posterior distributions often results in overly smoothed or blurry images, struggling to achieve the strict photorealism required for modern generative applications (Bredell et al., 2023; Hou et al., 2017).

To overcome the limitations of adversarial instability and restrictive variational bounds, the field witnessed a major paradigm shift toward continuous-time generative frameworks, most notably Diffusion Models (Denoising Diffusion Probabilistic Model (DDPM)) (Ho et al., 2020; Song et al., 2021). Diffusion models frame image generation as a gradual stochastic denoising process governed by Stochastic Differential Equations (SDEs), iteratively transforming a simple Gaussian prior into the complex data distribution by learning the score function of the perturbed data. While diffusion models have achieved unprecedented stability, scalability, and state-of-the-art image quality (Rombach et al., 2021; Saharia et al., 2022), their underlying probability paths often result in complex, highly curved generation trajectories. This curvature introduces the need for a significant number of trajectory discretization steps, making sampling computationally expensive (Liu et al., 2022; Lee et al., 2023b).

Recently, Flow Matching (Lipman et al., 2023; Liu et al., 2022; Albergo and Vanden-Eijnden, 2023) has emerged as a powerful reparameterization of diffusion models. By directly learning a deterministic continuous-time transport (velocity) field between noise and data, Flow Matching explicitly constructs straightened, simulation-free probability paths. This offers a framework that facilitates highly accurate sampling with significantly fewer integration steps, improving efficiency.

Beyond architectural improvements to the base models, optimizing pre-trained generative networks for specific downstream tasks has become a critical area of research. Initially, adapting diffusion models relied heavily on supervised, Parameter-Efficient Fine-Tuning (PEFT) (Houlsby et al., 2019) techniques. Methods such as DreamBooth (Ruiz et al., 2023) and CogView (Ding et al., 2021) were designed to incorporate novel concepts into the model generations using paired image-text data. To overcome the limitations of strictly supervised data, the field transitioned toward reward fine-tuning inspired by Reinforcement Learning from Human Feedback (RLHF) (Christiano et al., 2017) in language modeling. In the Large Language Model (LLM) domain, this post-training has become a standard component of the development pipeline,

largely standardizing around methods like Group Relative Policy Optimization (GRPO) (DeepSeek-AI et al., 2025; Shao et al., 2024) which reliably results in robust training. Conversely, establishing an equally reliable post-training paradigm remains a challenge for continuous-time frameworks like diffusion models.

Lee et al. (2023a) showed that diffusion models could be directly aligned with human preference by learning a reward function from binary human assessments and subsequently updating the generator via reward-weighted likelihood maximization. Concurrently, the introduction of ImageReward (Xu et al.) provided the first general-purpose text-to-image human preference reward model, trained on massive datasets of expert comparisons. By leveraging this robust reward scorer, the authors introduced Reward Feedback Learning (ReFL), an algorithm that directly optimized diffusion models at later denoising steps.

Despite the rapid advancement of these early reward alignment frameworks, current state-of-the-art approaches still face significant hurdles. First, aggressive fine-tuning against fixed proxy metrics often leads to reward hacking, where models produce unrealistic samples that score highly without reflecting genuine human preference (Eyring et al., 2025; Shen et al., 2025; Clark et al., 2024). Second, RL-based methods like GRPO suffer from a severe credit assignment problem; because the model receives only a terminal scalar reward, accurately attributing value to intermediate generative steps proves difficult (Parthasarathi et al., 2025; Mou et al., 2026; Tran et al., 2025). Third, Reinforcement Learning (RL)-based methods, such as recent GRPO-based approaches (Liu et al., 2025; He et al., 2025; Zheng et al., 2025) fundamentally struggle with training efficiency. Because these methods do not utilize the gradient of the reward, they must rely on multiple rollouts per prompt to estimate analytical gradients, which significantly slows down training convergence.

1.1 Research Questions

Ultimately, this thesis aims to answer the question:

*“Given a differentiable reward model,
what is the optimal way to reward fine-tune a diffusion model?”*

To answer this question, we introduce **Gladjoint Matching**, a novel approach designed to address the shortcomings of existing methods by combining the efficient trajectories of GLASS flows (Holderrieth et al., 2026b) with the Adjoint Matching (Domingo-Enrich et al., 2025) gradient-based fine-tuning method. To rigorously evaluate our proposed method and benchmark it against current state-of-the-art baselines, we investigate the following research questions:

RQ1 To what extent can the different fine-tuning methods mitigate or prevent reward hacking?

RQ2 How robust is each fine-tuning method under sparse reward settings?

RQ3 How effectively does each fine-tuning method improve the performance of the base model?

Through these research questions, we evaluate the most critical challenges prevalent in reward-based optimization and present our corresponding findings:

First, with the introduction of RL to enhance image generation models (Liu et al., 2025; Xue et al., 2025; He et al., 2025; Zheng et al., 2025), efficiency and optimization stability have become major concerns. We systematically investigate the prevalence of reward hacking and find that Gladjoint Matching successfully mitigates the severe overfitting and image degradation observed in earlier gradient-based methods, while avoiding the slow convergence and vulnerabilities we

expose in current RL-based approaches (**RQ1**). Secondly, in many practical settings the reward signal is sparse or only available for a subset of generations, we assess the robustness of fine-tuning methods under such conditions. Our experiments demonstrate that Gladjoint Matching is highly robust under sparse rewards, successfully improving the trained sparse reward, while achieving higher image quality than standard Adjoint Matching (**RQ2**). Lastly, since gradient-free approaches do not compute policy gradients and instead rely on multiple rollouts, the question arises if gradient-based optimization is fundamentally more efficient and effective for improving a text-to-image generation model when utilizing a specific reward. Through our general evaluation benchmarks we find that the gradient-based fine-tuning methods significantly outperform the RL-based methods under a fixed compute budget (**RQ3**).

1.2 Contributions

Through the investigation of these research questions, this thesis yields several important advancements. Specifically, our key contributions are as follows:

- We introduce Gladjoint Matching, demonstrating that it improves training convergence while achieving performance that achieves competitive performance while improving robustness and training efficiency over both the Gradient-based and RL-based baselines. Through extensive experiments, we demonstrate that our proposed method is more resilient to reward hacking and remains performant even when reward signals are sparse.
- We demonstrate that utilizing GLASS transitions maintains a memoryless process, which is needed for unbiased Adjoint Matching optimization.
- We provide a comprehensive comparative analysis of existing reward fine-tuning methods for diffusion models, evaluating their efficiency, robustness, and behavior in complex settings, thereby bridging a critical gap in literature by providing a comprehensive study of both RL-based and gradient-based reward fine-tuning methods.
- We show large vision-language models can be used as reward functions for gradient-based image generation fine-tuning methods, allowing for a significant increase in performance.

The remainder of this thesis is structured as follows. chapter 2 reviews theoretical background related to flow matching and image generation. chapter 3 provides an overview of the different fine-tuning methods utilized in this research. chapter 4 formulates the problem setup and introduces our improved method, while chapter 5 presents the experimental results. Finally, chapter 6 summarizes the main results.

Chapter 2

Background

This chapter establishes the theoretical foundation necessary for understanding flow matching and the fine-tuning of image-generation models. We begin by formalizing the mathematical notation and spaces used throughout this work. We then provide an overview of diffusion models, progressing to continuous normalizing flows and conditional flow matching. Subsequent sections detail GLASS flows and their role in improving computational efficiency, followed by an exploration of fine-tuning strategies and their associated reward functions. Finally, the chapter concludes with an explanation of classifier-free guidance, a technique used to enhance the quality of generated images.

2.1 Preliminaries

Before detailing the generative frameworks discussed in this chapter, we first establish the notation used throughout this work. Let our data instances (e.g., images) be represented as continuous variables residing in a d -dimensional Euclidean space, $x \in \mathbb{R}^d$. While some frameworks are traditionally formulated over discrete steps, we unify our definitions over a continuous time domain $t \in [0, 1]$. For reference, $t=1$ will always refer to the complex target data distribution and $t=0$ will refer to a tractable noise distribution (such as standard Gaussian noise). Let t represent a given timestep and t' represent a subsequent timestep closer to the data distribution ($0 \leq t < t' \leq 1$). We denote time-dependent probability density functions as $p_t(x)$. The time-dependent vector fields, which dictate the continuous evolution of data points over time, are denoted as $v_t(x) : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$. Finally, functions parameterized by a neural network with learnable weights θ are denoted with a superscript, such as v^θ or μ^θ .

2.2 Diffusion Models

DDPM (Ho et al., 2020) defines a generative framework which consists of two Markov chains: a forward process that gradually adds more noise to a datapoint, and a reverse process that removes noise with learned Gaussian transitions. Reframing the generative process over our continuous time domain $t \in [0, 1]$, the forward process from a cleaner state t' to a noisier state t is defined as

$$q(x_t|x_{t'}) = \mathcal{N}(x_t; \sqrt{1 - \beta_{t|t'}}x_{t'}, \beta_{t|t'}\mathbf{I}), \quad (2.1)$$

where $\beta_{t|t'}$ represents the variance scheduled for the interval between t' and t .

A defining feature of this forward process is that the marginal distribution at any intermediate time $t \in [0, 1]$ can be sampled in closed form. Defining the continuous signal schedule α_t

and noise schedule σ_t , the marginal state of a data point x_1 is given by:

$$q(x_t|x_1) = \mathcal{N}(x_t; \alpha_t x_1, \sigma_t^2 \mathbf{I}). \quad (2.2)$$

By design, at $t=1$, $\alpha_1 \approx 1$ and $\sigma_1 \approx 0$ (preserving the data), whereas at $t=0$, $\alpha_0 \approx 0$ and $\sigma_0 \approx 1$ (resulting in a standard Gaussian distribution $\mathcal{N}(0, \mathbf{I})$). Consequently, the noisy state can be reparameterized as

$$x_t = \alpha_t x_1 + \sigma_t \epsilon, \quad (2.3)$$

where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, with α_t and σ_t representing the signal and noise scales, respectively. Starting from pure noise $x_0 \sim p_0$, the model learns to invert the destructive Markov transitions, stepping from the current state x_t to a cleaner state $x_{t'}$, where the mean is parameterized by a neural network:

$$p^\theta(x_{t'}|x_t) = \mathcal{N}(x_{t'}; \mu^\theta(x_t, t), \sigma_{t|t'}^2 \mathbf{I}). \quad (2.4)$$

Rather than directly predicting the mean μ^θ , Ho et al. (2020) demonstrated that optimizing the variational lower bound can be simplified into a regression task. The network is trained to predict the noise ϵ added to the data at any uniformly sampled continuous time $t \in [0, 1]$:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t \sim \mathcal{U}(0,1), x_1 \sim p_1, \epsilon} [\|\epsilon - \epsilon^\theta(x_t, t)\|^2]. \quad (2.5)$$

Taking the limit as the step size between t and t' approaches zero, these discrete Markov chains converge to a continuous-time Stochastic Differential Equation (SDE) (Song et al., 2021).

2.3 Continuous Normalizing Flows

Introduced by Chen et al. (2018), Continuous Normalizing Flows (CNFs) provide a framework for transforming a simple prior distribution p_0 into a more complex target distribution p_1 via a continuous-time evolution of probability densities. In particular, p_0 and p_1 correspond to two instances along a probability density path

$$p : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}_{>0},$$

where, for each time $t \in [0, 1]$, $p_t(x)$ defines a valid probability density satisfying

$$\int p_t(x) dx = 1.$$

Data points $x = (x^1, \dots, x^d) \in \mathbb{R}^d$ evolve according to the Ordinary Differential Equation (ODE):

$$\frac{d}{dt} \phi_t(x) = v_t(\phi_t(x)), \quad (2.6)$$

where $v : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a time-dependent vector field, and $\phi : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ denotes the associated flow that transports samples from p_0 to p_1 . Chen et al. (2018) propose to model v utilizing a neural network, where its parameters $\theta \in \mathbb{R}^p$ are learned during training.

2.4 Conditional Flow Matching

Three works introduced the idea of Conditional Flow Matching (CFM), each proposing a slightly different framework: conditional matching (Lipman et al., 2023), rectified flows (Liu et al., 2022), and stochastic interpolants (Albergo and Vanden-Eijnden, 2023). In this work, we follow the CFM framework of Lipman et al. (2023). Similar to CNFs, CFM aims to find a probability

path p , which allows us to flow from p_0 to p_1 . In CFM, this is done utilizing a ‘ground truth’ conditional vector field $u_t(x|x_1)$, where x_1 is distributed according to some data distribution q , for which we do not have access to the density function. This allows for the following objective, to train the CNF:

$$\mathcal{L}_{CFM} = \mathbb{E}_{t,x_1,p_t(x|x_1)} [\|v_t^\theta(x) - u_t(x|x_1)\|^2]. \quad (2.7)$$

This can intuitively be seen as a regression of the time-dependent vector field $v_t^\theta(x)$ towards a simple per-sample conditional target $u_t(x|x_1)$. While the true marginal vector field that is required to transition between distributions is typically intractable due to the unknown data density, the Conditional Flow Matching objective bypasses this by leveraging the fact that the gradient of the marginal loss is equivalent to the expectation of the conditional loss.

2.5 Inference and Sampling

While the previous sections establish the training objectives used to align the model’s predictions with a target vector field or noise schedule, we now introduce the inference process. During training, the model learns, for instance, a time-dependent vector field $v^\theta(x, t)$ by regressing against a known forward trajectory. During inference, however, the model must rely on this learned approximation to simulate a full trajectory, starting from a sample in the pure noise prior $x_0 \sim p_0$ at $t = 0$ and transporting it to a generated data sample $x_1 \sim p_1$ at $t = 1$. This generation is either done using an ODE or SDE.

Prior to formalizing these dynamics, it is important to note the existence of various model parameterizations, such as directly predicting the clean data x_1 , the noise $\epsilon^\theta(x_t, t)$, the score $s^\theta(x_t, t)$, or the velocity $v^\theta(x_t, t)$. Crucially, because flows and diffusions share an underlying mathematical structure, we only need to learn one of these objects in practice. For instance, there is a direct, linear relationship between the score and the velocity (Ma et al., 2024):

$$s(x, t) = \sigma_t^{-1} \frac{\alpha_t v(x, t) - \dot{\alpha}_t x}{\dot{\alpha}_t \sigma_t - \alpha_t \dot{\sigma}_t}. \quad (2.8)$$

We continue by utilizing the velocity prediction framework. Under this formulation, sample generation can be executed via the probability flow ODE (Song et al., 2021), governed directly by the velocity field:

$$\frac{d}{dt}x_t = v^\theta(x_t, t), \quad (2.9)$$

where v^θ represents the model’s predicted velocity. ODE sampling is highly advantageous for several reasons. It establishes a deterministic, one-to-one mapping between the noise distribution and the data distribution, which preserves semantic structures during latent interpolation. Furthermore, it enables the use of fast, higher-order numerical ODE solvers, allowing for high-quality generation in fewer discretization steps. (Chen et al., 2018).

Alternatively, the sampling process can be formulated as an SDE (Ma et al., 2024), which injects stochasticity into the generative trajectory through a Brownian motion W_t :

$$dx_t = \left[\left(1 + \frac{\alpha_t g(t)^2}{2\sigma_t} \right) v^\theta(x_t, t) - \left(\frac{g(t)^2 \dot{\alpha}_t}{2(\dot{\alpha}_t \sigma_t - \alpha_t \dot{\sigma}_t)} \right) x_t \right] dt + g(t) dW_t, \quad (2.10)$$

where $g(t)$ controls the diffusion scale, and $\dot{\alpha}_t$ and $\dot{\sigma}_t$ denote the time derivatives of the signal and noise scales, respectively. This inherent stochasticity is mathematically critical for alignment and fine-tuning. Because an SDE transition yields a Gaussian distribution rather than a deterministic point, it enables the computation of step-wise objectives, such as the Kullback-Leibler (KL) divergence (Kullback and Leibler, 1951) between a fine-tuned policy and a base model. Furthermore, it improves exploration during fine-tuning (Sheng et al., 2025). However,

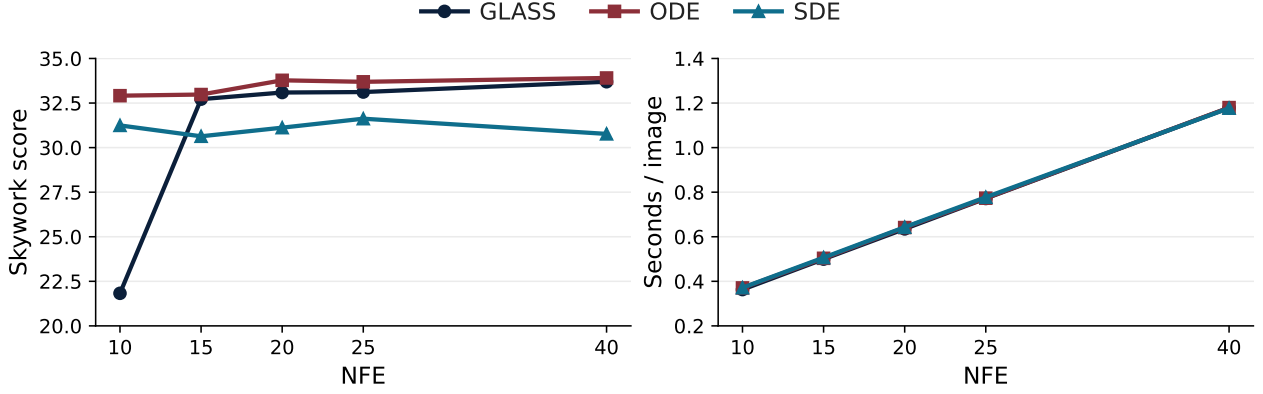


Figure 2.1: Performance comparison of GLASS, ODE, and SDE transitions. GLASS flows achieve image quality superior to SDEs and comparable to ODEs, preserving stochasticity without increasing generation time.

this injected noise requires a significantly higher number of Neural Function Evaluations (NFEs) to maintain numerical stability, inherently slowing down the generation process.

2.6 GLASS Flows

A significant limitation of current flow matching fine-tuning methods is their reliance on SDE-based sampling. Because this approach introduces noise at every discretization step, the sampling process becomes computationally intensive and inherently slower. Furthermore, traditional ODE sampling often proves unsuitable for adaptation methods that require a defined model likelihood or KL-divergence, as the deterministic nature of the backward ODE path lacks the stochasticity necessary for such probabilistic objectives.

GLASS Flows (Holderrieth et al., 2026b) addresses this challenge by framing the transition between two time steps as a conditional generation problem. By learning an “inner flow” that maps the current state and time to a future state, GLASS Flows enables efficient ODE transitions while keeping the stochasticity.

GLASS flows specifically define the inner flow matching model $u_s(\bar{x}_s|x_t, t)$ where s is the new inner time variable ($0 \leq s, t \leq 1$). This gives an objective that conditions on the current outer time variable t and current state x_t , and allows us to sample $x_{t'} \sim p_{t'|t}(x_{t'}|x_t)$ ($t' > t$).

These inner flows, or ‘GLASS flows’ are defined by:

$$p_{t'|t}(x_{t'}|x_t) = \frac{p_{t,t'}(x_t, x_{t'})}{p_t(x_t)} = \int p_{t'|t}(x_{t'}|x_t, z) p_{1|t}(z|x_t) dz, \quad (2.11)$$

where the transition kernel conditioned on a specific clean data point z is defined by the gaussian:

$$p_{t'|t}(x_{t'}|x_t, z) = \mathcal{N}(x_{t'}; \bar{\alpha}z + \bar{\gamma}x_t, \bar{\sigma}^2 I_d), \quad (2.12)$$

and where the parameters are defined by:

$$\bar{\gamma} = \rho \sigma_{t'} \sigma_t^{-1}, \quad \bar{\alpha} = \alpha_{t'} - \bar{\gamma} \alpha_t, \quad \bar{\sigma}^2 = \sigma_{t'}^2 (1 - \rho^2).$$

The second part of GLASS flows is the standard flow matching posterior $p_{1|t}(z|x_t)$, which introduces the stochasticity. The parameter ρ defines the correlation across time, controlling the similarity between x_t and $x_{t'}$.

Intuitively, GLASS Flows can be thought of as simulating a “flow matching model within a flow matching model.” Rather than inefficiently injecting noise at every step like what is done

with SDEs, it treats the outer step transition as its own conditional generation task. It picks a random starting point for this “inner” transition to guarantee stochasticity, and continues with an ODE to maintain speed and efficiency.

As demonstrated in Figure 2.1, the use of GLASS flows improves the quality of generated images. The figure illustrates that applying GLASS transitions during image generation yields higher Skywork VLM scores (detailed further in subsection 5.1.2) compared to standard SDE transitions. Furthermore, by utilizing fewer NFEs in the generation trajectory, we achieve similar Skywork scores while considerably reducing the overall image generation time.

2.7 Fine-Tuning Flow Matching Models & VLMs

Fine-tuning in diffusion and flow matching models is aimed at learning the tilted distribution (Domingo-Enrich et al., 2025):

$$p^*(x) \propto p^{base}(x) \exp\left(\frac{1}{\beta}r(x)\right), \quad (2.13)$$

where p^{base} is the base pre-trained generative model’s sample distribution, $r(\cdot)$ is any reward function, and β is a parameter controlling the reward scale. As shown by (Eyring et al., 2025), this definition can be rewritten into a KL divergence:

$$\min_{\theta} D_{KL}(p^{\theta} \parallel p^*) = \min_{\theta} \left[D_{KL}(p^{\theta} \parallel p^{base}) - \frac{1}{\beta} \mathbb{E}_{x \sim p^{\theta}}[r(x)] \right], \quad (2.14)$$

where p^{θ} is the distribution generated by the optimized model. This definition highlights how the fine-tuned distribution should still have the same support as the original base distribution and thus should not change drastically. Furthermore, these definitions show that decreasing β increases the scale of the reward function and thereby increases the divergence between the tilted distribution and the base distribution. Such scaling is common in fine-tuning methods: it is typically parameterized by β in GRPO-based approaches and by λ in gradient-based optimization. In GRPO-style methods, β usually appears in the loss as a scaling factor on the KL divergence (Kullback and Leibler, 1951) between the optimized model and the base model, whereas in gradient-based methods λ scales either the reward itself or its gradient. We change the scaling of the reward gradient in the gradient-based methods, into a scaling of the regularization term of the loss, as discussed in section 4.1.

Early alignment methods often utilized RLHF (Christiano et al., 2017; Xu et al.), which relied on gathering large-scale human preference data. Consequently, modern approaches employ Vision-Language Models (VLMs) as scalable and automatic proxies for human preference. Unlike early contrastive models such as CLIP (Radford et al., 2021), which only measure global text-image similarity, modern VLMs are capable of complex visual reasoning, attribute binding, and spatial understanding (Wang et al., 2024, 2025b). These models can output a scalar reward either by directly regressing a preference score or by extracting the log-probabilities of specific evaluation tokens that indicate high visual quality or prompt alignment.

2.8 Classifier-Free Guidance

To improve conditioning of the generative process on textual prompts, Classifier Guidance (Dhariwal and Nichol, 2021) was introduced. In this approach, an auxiliary classification model is trained to predict labels from noisy states x_t . The gradients from this classifier are then used to steer the sampling trajectory toward images of a specific label. To eliminate the need for



Figure 2.2: Improved image quality when using CFG for the prompts: “A cozy reading nook with a velvet armchair, soft blanket, stack of books, and rain against the window”, and “A laptop on top of a teddy bear.”

this separate classifier, Classifier-free Guidance (CFG) (Ho and Salimans, 2022) implicitly incorporates guidance directly into the training of the image generator. Rather than training two distinct models, a single model learns both conditional and unconditional distributions by randomly dropping the conditioning signal with a fixed probability during pretraining. At inference time, a linear combination of the conditional and unconditional predictions is used to produce the final guided velocity $\tilde{v}_t$:

$$\tilde{v}_t(x|c) = w \cdot v_t(x|c) + (1 - w) \cdot v_t(x|\emptyset), \quad (2.15)$$

where c is the conditioning prompt and w represents the guidance scale. The term $v_t(x|\emptyset)$ denotes the unconditioned velocity, obtained by replacing the prompt with an empty string. While CFG significantly improves the visual quality of generated images, as seen in Figure 2.2, it introduces a computational overhead by requiring two separate forward passes of the model at each sampling step.

Having established the foundations of flow matching, fine-tuning, and guidance, the following chapter will explore the broader landscape of related work, focusing specifically on how recent alignment methods navigate the inherent computational overhead and efficiency bottlenecks of these generative processes.

Chapter 3

Related Work

This chapter contextualizes our proposed approach within the broader landscape of reward-based alignment for generative models. As discussed in section 2.7, reward-based fine-tuning aims to steer a base model toward a reward-tilted distribution. Existing methods differ mainly in how they approximate this target distribution and how they obtain optimization signals for updating the model.

We organize the literature into two broad categories. *Gradient-based* methods, including Direct Reward Fine-Tuning (DRaFT) (Clark et al., 2024), Direct Align (Shen et al., 2025), and Adjoint Matching (AM) (Domingo-Enrich et al., 2025), which directly utilize the gradient of the reward function to improve the base model. This specific utilization of this gradient makes them sample-efficient, but also more susceptible to reward hacking (Shen et al., 2025). Conversely, *RL-based* approaches, such as FlowGRPO (Liu et al., 2025), DiffusionNFT (Zheng et al., 2025), and TempFlow-GRPO (He et al., 2025), avoid backpropagating through the full generation process by estimating advantages from sampled rollouts. These methods are more flexible with respect to black-box or non-differentiable rewards, but require multiple generations per prompt and can therefore be less efficient at fitting reward preferences.

3.1 Gradient-based Optimization

Adjoint Matching. Gradient-based optimization methods fundamentally struggle with the high memory cost of backpropagating through a fully unrolled multi-step generation process (Clark et al., 2024). AM (Domingo-Enrich et al., 2025) bypasses this trajectory bottleneck by casting the SOC problem (Bellman and Dreyfus, 2015) as a regression task onto a target vector field. It simulates trajectories from the current process and simultaneously solves an adjoint process backwards in time. This backward process yields a regression target that acts as a stochastic estimator of the optimal control. The model is then updated to match this target via a simple least-squares objective.

A critical contribution of this method is the introduction of the *lean* adjoint state. Traditional continuous adjoint methods (Pontryagin, 1987) require computing the Jacobian of the control, which is computationally expensive and can increase variance. However, because these Jacobian-related terms have an expectation of zero at the optimal solution, they are excluded.

Domingo-Enrich et al. (2025) also show that utilizing a memoryless noise schedule is needed to get rid of the initial value function bias and correctly optimize towards the tilted distribution. By injecting a sufficient amount of noise during the forward process, the initial noise distribution and the target image become statistically independent. Consequently, we can factorize their joint probability as follows:

$$p^{\text{base}}(x_0, x_1) = p^{\text{base}}(x_0)p^{\text{base}}(x_1). \quad (3.1)$$

This independence removes the initial value function bias, allowing for correct training toward the reward-tilted distribution. Without this independence, the bias would introduce an unwanted value function term, evaluated on the initial distribution, into the optimized objective. **DRaFT.** DRaFT (Clark et al., 2024) is one of the earliest gradient-based reward fine-tuning methods, originally introduced for diffusion models. It relies on backpropagating the reward gradient directly through the unrolled, multistep diffusion sampling chain. While conceptually straightforward, unrolling the full sampling process scales poorly: it consumes massive amounts of memory. The original authors overcome this issue by truncating the backpropagation through only the last sampling step, which significantly speeds up training but does introduce significant reward hacking (Shen et al., 2025). Importantly, DRaFT does not incorporate any mechanism to remain close to the base model during fine-tuning, which can lead to substantial divergence from the base model over the course of training. The authors introduce three main variants based on the truncation depth. DRaFT- K truncates backpropagation to only the last K sampling steps to balance memory savings, optimization stability, and gradient accuracy. DRaFT-1 differentiates solely through the final denoising step. DRaFT-LV is a low-variance extension of DRaFT-1 that reduces gradient variance by averaging estimates over multiple independent noise samples at the final step.

Direct Align and SRPO To address the limitations of DRaFT, SRPO (Shen et al., 2025) builds on Direct-Align, which utilizes a single-step recovery shortcut. By using a predefined noise prior, Direct-Align estimates clean images from highly noisy intermediate steps without unrolling the full trajectory. While this effectively reduces computation, it remains an approximation of the true diffusion dynamics.

The single-step recovery in Direct Align is done by injecting ground-truth Gaussian noise into a clean image to place it at a chosen timestep, producing a sequence of noisy states. Each noisy state is denoised in one step, and the resulting intermediate rewards are aggregated using a decaying discount factor, where later timesteps are down-weighted to mitigate reward hacking at higher frequency stages. SRPO builds on Direct Align by utilizing the reconstruction and exact recovery into a new text-conditioned reward. Because SRPO requires a prompt-conditioned reward model, we evaluate only Direct Align.

3.2 RL-based Optimization

The reinforcement learning methods used for fine-tuning bypass the need for differentiable reward functions by relying on the REINFORCE algorithm (Williams, 1992; Sutton et al., 1999). By using the so-called characteristic eligibility, the gradient of the expected reward can be rewritten as:

$$\nabla_{\theta} \mathbb{E}_{x \sim p^{\theta}} [r(x)] = \mathbb{E}_{x \sim p^{\theta}} [r(x) \nabla_{\theta} \log p^{\theta}(x)]. \quad (3.2)$$

This formulation allows the model to update its parameters θ purely by sampling from its own generations x and scaling their log-probabilities by the scalar reward $r(x)$. Because the gradient is not required to pass through $r(x)$ itself, RL can seamlessly optimize against black-box or non-differentiable metrics.

FlowGRPO Introduced by Liu et al. (2025), FlowGRPO is the first work to introduce the usage of GRPO (Shao et al., 2024) for flow matching models. Originally developed to enhance the reasoning capabilities of LLMs (DeepSeek-AI et al., 2025), GRPO offers significant memory efficiency over standard policy gradient methods like PPO (Schulman et al., 2017) by eliminating the need for a separate value network. Instead, it introduces a group-relative formulation: it generates a group of outputs for a given prompt and estimates the advantage by normalizing the rewards relative to that specific group. FlowGRPO adapts this framework by evaluating reverse-time trajectories rather than discrete tokens, allowing it to quantify relative improvement across

a group of rollouts.

TempFlow. Improving on FlowGRPO, TempFlow (He et al., 2025) introduces a reweighting of the policy loss by the noise level at each timestep. This modification increases the contribution of earlier, noisier timesteps to the overall loss. The authors argue that these early steps should be emphasized because they correspond to a larger exploration space, whereas later timesteps perform only fine-grained refinements and therefore warrant comparatively less weight. Furthermore, the authors employ trajectory branching to further improve efficiency. In this approach, the system follows a deterministic trajectory up to a branching time step (k), at which point it is perturbed once using an SDE; from there, it again evolves deterministically (ODE–SDE–ODE). The terminal reward is then reassigned to step (k), thereby localizing credit assignment to the noise injection at that specific time step.

DiffusionNFT. Instead of defining GRPO on the reverse-time SDE, DiffusionNFT (Zheng et al., 2025) defines it on both the forward ODE and SDE, allowing for efficient training and allowing data collection with any solver. To stabilize training, DiffusionNFT optimizes a single loss that uses the sample’s reward to interpolate between a *positive* direction pulling the model toward the target flow v and a *negative* direction pushing it away. Importantly, these directions do not come from separate networks but are linear combinations of v^θ and v^{old} .

Chapter 4

Gladjoint Matching

Building upon the theoretical foundations of Flow Matching and addressing the computational and performance limitations of existing fine-tuning paradigms, this chapter introduces our novel approach: Gladjoint Matching. Our method integrates the more efficient GLASS trajectories into the Adjoint Matching fine-tuning framework. We demonstrate that this approach maintains the memoryless noise schedule required by Adjoint Matching.

We propose **Gladjoint Matching (GM)**, which uses GLASS transitions as an efficient sampler inside Adjoint Matching fine-tuning. The goal is not to define a new fine-tuning objective, but to replace the expensive SDE trajectory sampler used in AM with a faster transition sampler while preserving the memoryless noise schedule required by AM. As demonstrated in Figure 2.1, utilizing GLASS transitions yields higher-quality images than standard SDE trajectories with the same amount of discretization steps.

We show that, under an exact transition-sampling assumption, choosing the GLASS correlation parameter appropriately makes the GLASS transition kernel coincide with the memoryless noise schedule used by AM. The memoryless noise schedule requirement of AM ensures independence between the initial noise and the generated sample, giving the simplification found in Equation 3.1. Domingo-Enrich et al. (2025) show that utilizing this memoryless noise schedule is sufficient to remove the initial value-function bias, which allows the SOC solution with $f=0$ and $g=-r$ to optimize towards the reward-tilted distribution.

We now show that using GLASS transitions preserves this condition under an exact transition assumption. Let

$$P_{t'|t}^{\text{memoryless}}(\cdot | x_t) \quad (4.1)$$

denote the transition kernel of the memoryless process used by AM. Separately, GLASS defines a family of transition kernels by specifying a joint coupling between x_t and $x_{t'}$ conditioned on the clean variable z . We denote the corresponding GLASS transition kernel by:

$$P_{t'|t}^{\text{GLASS},\rho}(\cdot | x_t), \quad (4.2)$$

where ρ controls the correlation between the noise variables at times t and t' . For the particular choice:

$$\rho_{t,t'} = \frac{\alpha_t \sigma_{t'}}{\sigma_t \alpha_{t'}}, \quad (4.3)$$

the GLASS transition family recovers the DDPM transition kernel (Holderrieth et al., 2026b). Since this DDPM transition corresponds to the memoryless transition used in AM, we obtain

$$P_{t'|t}^{\text{GLASS},\rho_{t,t'}}(\cdot | x_t) = P_{t'|t}^{\text{mem}}(\cdot | x_t). \quad (4.4)$$

Thus, in GM, GLASS is used as an efficient inner-flow sampler for the same memoryless transition law on which AM is based.

Functionally, the choice of ρ is what aligns the efficiency of GLASS transitions with the requirements of Adjoint Matching. For instance, if ρ were set to 1, the conditional transition between x_t and $x_{t'}$ would become entirely deterministic.

4.1 Regularization

The original implementation of AM utilized a scaling of the reward gradient $\nabla_x g(x_1)$ to achieve convergence to a tilted distribution with a higher influence of the reward model. We instead replace this scaling with a downscaling of the regularization term in the loss. This reparameterization does not change the trade-off between reward maximization and proximity to the base model. Instead, it expresses the trade-off in the same direction as the KL coefficient used in RL-based methods: smaller values of λ/β allow larger deviations from the base model, whereas larger values keep the fine-tuned model closer to the base model. This makes the amount of regularization easier to compare across AM, GM, and the RL-based baselines. This change results in the final loss used by both AM and GM:

$$\mathcal{L}_{\text{AM-GM}} = \sum_{t \in \{0, \dots, 1-h\}} \left\| \lambda \frac{\sigma(t)}{2} (v^{\text{finetune}}(x_t, t) - v^{\text{base}}(x_t, t)) + \sigma(t) \tilde{a}_t \right\|^2. \quad (4.5)$$

Furthermore, utilizing this downscaling of regularization instead of upscaling the reward gradient reduces overfitting while keeping training speed the same, as seen in Appendix A.

4.2 Timestep Subsampling and Endpoint-Level Loss

To improve the efficiency of AM, the original authors introduced a subsampling strategy for backpropagating through the term $\left\| \frac{2}{\sigma(t)} (v^{\text{finetune}}(x_t, t) - v^{\text{base}}(x_t, t)) + \sigma(t) \tilde{a}_t \right\|^2$. Rather than using the entire trajectory, they restrict sampling to the first $T-1$ steps (up to $1-h$): half of the required steps are drawn uniformly from the initial 60% of this interval, and the remaining half are drawn uniformly from the final 40%. We adopt this same subsampling for GM and perform ablations on this number of utilized steps k in both AM and GM, found in subsection 5.2.2.

In addition to timestep subsampling, we evaluate the GM loss at the level of complete GLASS transitions rather than at the inner states. GLASS samples a transition from x_t to $x_{t'}$ by introducing an inner flow with its own time variable s (Holderrieth et al., 2026b), evolving an auxiliary state $\bar{x}_s$ and returning the endpoint $\bar{x}_{s=1}$ as the sample from the transition kernel:

$$\bar{x}_{s=1} \sim p_t^{\text{GLASS}, \rho}(\cdot \mid x_t). \quad (4.6)$$

The key distinction is that the inner states $\bar{x}_s$ are not states of the original generative trajectory at outer time t or t' . Instead, they are auxiliary samples from the GLASS bridge posterior used to solve the conditional transition-sampling problem. More explicitly, GLASS constructs an inner probability path of the form:

$$p_s^{\text{GLASS}, \rho}(\bar{x}_s \mid x_t) = \int p_s(\bar{x}_s \mid x_t, z) p_{1|t}(z \mid x_t) dz, \quad (4.7)$$

where $p_s(\bar{x}_s \mid x_t, z)$ is a Gaussian bridge path conditioned on the current outer state x_t and a clean latent variable z , while $p_{1|t}(z \mid x_t)$ is the flow-matching posterior over clean samples. Thus, the distribution followed by the inner states is an auxiliary density over the inner time variable s , not the original marginal density p_t . This specifically matters for the Adjoint Matching objective, because its loss term is evaluated at a state $x_t \sim p_t$, where the model velocity

$v^\theta(x_t, t)$, the base velocity $v^{\text{base}}(x_t, t)$, the diffusion coefficient $\sigma(t)$, and the adjoint target $\tilde{a}_t$, all correspond to the same outer time variable. However, GLASS only guarantees the endpoint $\bar{x}_{s=1}$ has the desired transition law.

For this reason, applying the AM loss to the GLASS inner states would change the training distribution: the model would be regularized and matched on samples from a mismatched trajectory. Therefore, in GM, the inner GLASS states are treated as part of the transition sampler, while the AM objective remains defined on the outer discretization.

Chapter 5

Experiments

This chapter presents numerical experiments designed to evaluate the RL- and gradient-based methods, together with our proposed Gladjoint Matching method. First, we provide a general overview of the experimental setup and the benchmarks used. Next, we perform synthetic evaluations to test the robustness of the training methods against reward hacking and sparse rewards. The reward hacking evaluation consists of two experiments: the first tests a basic setting where models are trained simply to increase the number of red pixels in the output image, while the second focuses on a more realistic reward function combined with a deliberately small training set. Finally, we detail the method’s overall performance on two general evaluation benchmarks: GenEval and UniGenBench++.

5.1 Experimental Settings

5.1.1 Evaluation Benchmarks

To rigorously assess the prompt alignment, compositional accuracy, and overall generation, the fine-tuned models are evaluated across two comprehensive text-to-image benchmarks: GenEval (Ghosh et al., 2023) and UniGenBench++ (Wang et al., 2026).

GenEval. GenEval is utilized to evaluate the compositional capabilities of the generated images. It systematically tests a model’s ability to accurately reflect specific prompt elements across several distinct types of prompts: single object generation, two objects, counting (generating the correct number of instances), color attribution, spatial positioning, and attribute binding. By measuring these capabilities, GenEval allows for the quantification of fundamental degradation of compositional skills or possible overfitting when utilizing specific fine-tuning methods. GenEval performance is measured with the Mask2Former (Cheng et al., 2022) segmentation model for object detection, and CLIP ViT-L/14 (Radford et al., 2021) for color classification. GenEval includes a test- and train-set with 553 and 50,000 entries, respectively.

UniGenBench++. To provide a more holistic evaluation of the generated outputs, we employ UniGenBench++. This benchmark extends beyond basic composition to assess a wider array of visual and semantic capabilities, including style adherence, world knowledge, complex actions, logical reasoning, grammar, and text rendering. Following the original implementation we utilize their 32B UniGenBench VLM as an automatic evaluator. UniGenBench++ consists of a 600-entry test set but lacks a training set. Therefore, following Eyring et al. (2025), we compile a training-set of approximately 70K entries by combining T2I-Compbench (Huang et al., 2023), ABC-6K (Feng et al., 2022), and Pick-a-Picv2 (Kirstain et al., 2023).

5.1.2 Implementation Details

We fine-tune Stable Diffusion 3.5-Medium (SD3.5-M) (Esser et al., 2024) using LoRA (Hu et al., 2021) weights with rank 32 and alpha 64. All models are trained using the AdamW optimizer (Loshchilov and Hutter, 2019) with a learning rate of 1×10^{-4} and a global batch size of 16. Following Liu et al. (2025); He et al. (2025); Zheng et al. (2025); Choi et al. (2026), images are generated at a resolution of 512×512 . For all methods except DiffusionNFT, a CFG scale of 4.5 is used, following their original implementations. During inference, we sample four random initial seeds per prompt, resulting in four generated images per text input.

Training Dynamics. To ensure a fair comparison, all training methods except AM and GM utilize global gradient clipping to a norm of 1. The gradients for the alignment and regularization components of the AM and GM losses are clipped separately to achieve closer parity with DRaFT and Direct Align. Preliminary results indicate that this also improves convergence speed. We omit the usage of Exponential Moving Average (EMA) (Huang et al., 2017) in the synthetic settings of section 5.2 and section 5.3. This omission significantly accelerates training and enables a clearer analysis of the training dynamics. However, we do employ EMA with a decay of $\eta=0.999$ for the general preference experiments evaluated in section 5.4 to improve stability. The number of discretization steps within the training trajectory serves as a crucial hyperparameter for the training methods. We investigate its impact through an ablation study presented in subsection 5.2.2. Unless otherwise specified, the RL-based methods employ 10 discretization steps, whereas gradient-based approaches utilize 40 steps, consistent with prior research. Additional training hyperparameters are listed in Appendix B.

Training Rewards. For general preference reward training on GenEval and UniGenBench++, we use NVILA-Lite-2B-Verifier¹ (Liu et al., 2024), a VLM fine-tuned specifically to serve as a verifier for text-to-image generation. Following the original implementation, the reward is obtained from the probability of the 'yes' token, conditioned on the generated image and an input question containing the prompt.

To measure reward-hacking, we use both ImageReward (Xu et al.) and Skywork-VL-Reward-7B² (Wang et al., 2025b). ImageReward uses BLIP (Li et al., 2022) as its backbone and is fine-tuned on human preference data. Skywork leverages Qwen-2.5-VL as its backbone, with an added fine-tuned reward head that outputs a scalar in the range $[-50, 50]$.

5.2 Reward Hacking

This section addresses RQ1: *to what extent can the different fine-tuning methods mitigate or prevent reward hacking?* We evaluate this question in two settings. First, we use a synthetic redness reward, which is intentionally easy to exploit and therefore exposes severe reward hacking behavior. Second, we use the NVILA verifier as a more realistic prompt-dependent reward model.

5.2.1 Redness

This experiment answers RQ1 in a deliberately simplified setting. To this end, we use a simple "redness" reward, which measures the proportion of red pixels in the generated images and rescales this value to lie between 0 and 1. Crucially, this represents a convex reward function, yielding a smooth optimization landscape with well-behaved gradients. This reward simulates a setting in which the optimization focuses on a single visual attribute of the image, while overall

¹<https://huggingface.co/Efficient-Large-Model/NVILA-Lite-2B-Verifier>

²<https://huggingface.co/Skywork/Skywork-VL-Reward-7B>

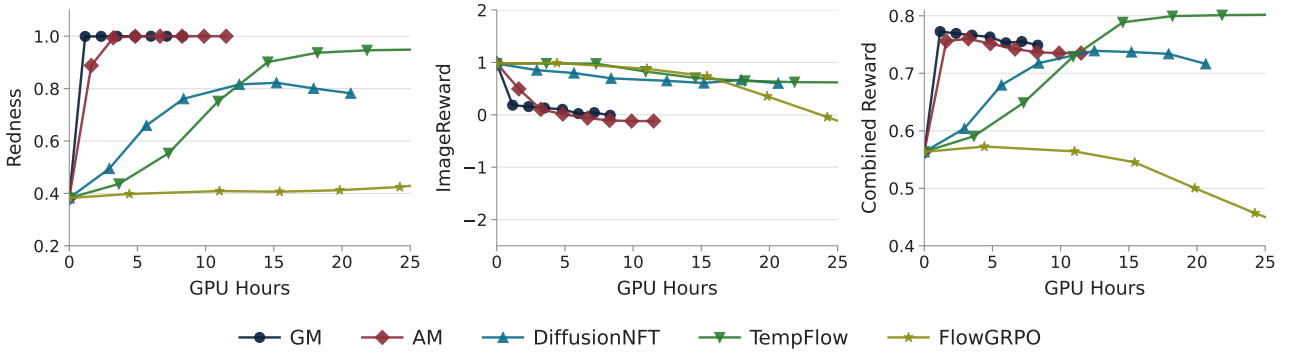


Figure 5.1: Redness and ImageReward throughout training for **Adjoint Matching**, **Gladjoint Matching** and the **RL** fine-tuning methods. The combined plot shows redness together with ImageReward normalized to the range $[0, 1]$. Gladjoint Matching and Adjoint Matching achieve significantly faster convergence than the RL-based methods.

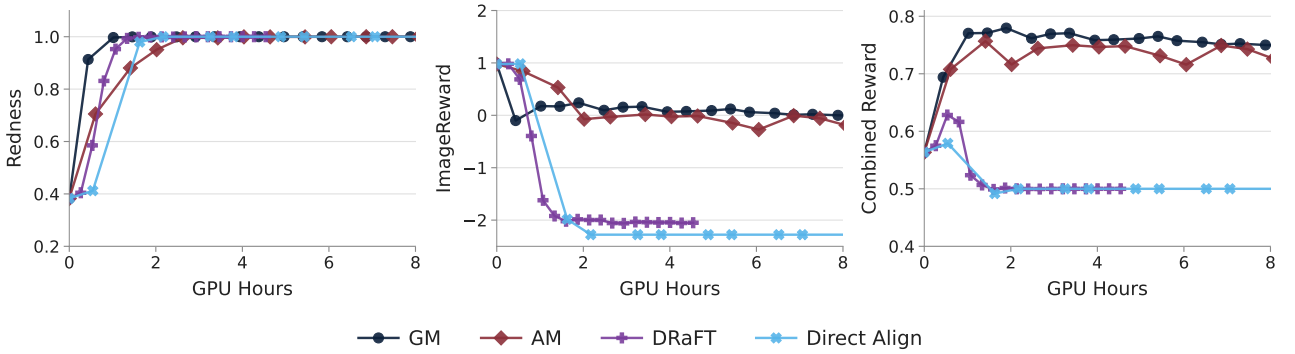


Figure 5.2: Redness and ImageReward throughout training for the **gradient-based** fine-tuning methods. The combined plot shows redness together with ImageReward normalized to the range $[0, 1]$. Gladjoint Matching achieves faster convergence and less overfitting compared to the other gradient-based methods.

image fidelity is ignored. As a result, highly noisy or even entirely red images can achieve very high reward scores despite being of poor visual quality. To investigate the occurrence of reward hacking under a redness-based reward function, we use a small training set of 50 samples and an evaluation set of 20 samples. The prompts are high-level and each describes only a single object.

Figure 5.1 shows that the gradient-based methods achieve significantly higher efficiency than the RL fine-tuning approaches when optimizing for a simple reward function, such as redness. Both GM and AM increase redness rapidly, whereas the RL-based approaches require substantially more training before showing comparable performance. Furthermore, Figure 5.2 shows that GM achieves faster convergence than Adjoint Matching, DRaFT and Direct Align, while overfitting less. This suggests that, in this simplified setting, GM provides the best trade-off between reward optimization and general image quality.

The role of regularization is further illustrated in Figure 5.3. For Adjoint Matching, changing the regularization strength shifts the model toward different points on the trade-off between maximizing redness and preserving image quality. Stronger regularization keeps the fine-tuned model closer to the base model and leads to higher ImageReward, whereas weaker regularization allows the model to increase redness more aggressively. Corresponding plots illustrating the effect of regularization for the other methods can be found in Appendix C.

We further support the quantitative findings with a qualitative comparison in Figure 5.4 and Figure 5.5, which shows generated samples from a fixed initial noise at different training stages.

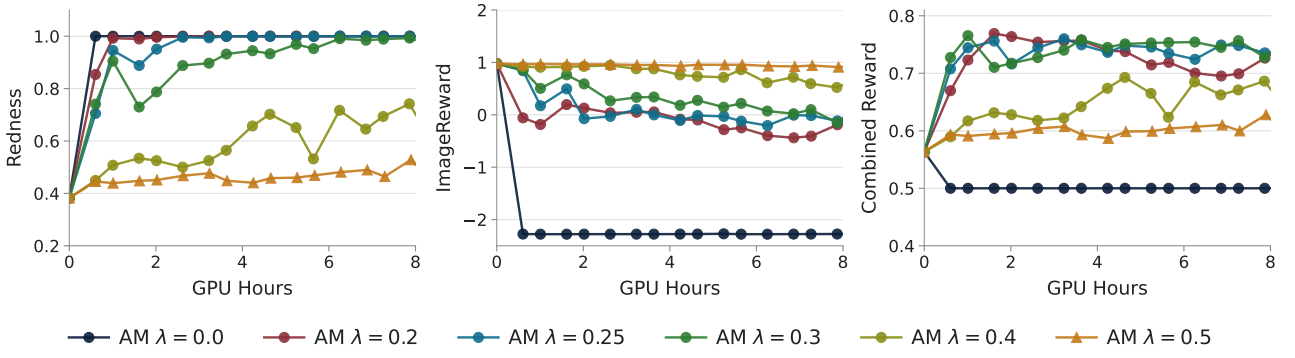


Figure 5.3: Influence of different regularization on the training of Adjoint Matching. Adjusting the regularization parameter consistently controls the trade-off between maximizing the target reward and maintaining proximity to the base model.

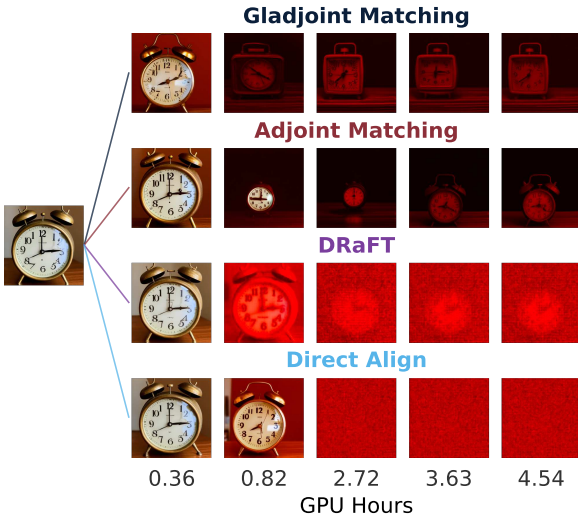


Figure 5.4: Generated images throughout training with **gradient-based** fine-tuning methods. DRaFT and Direct Align overfit significantly, while Gladjoint Matching and Adjoint Matching successfully remain close to the base model.

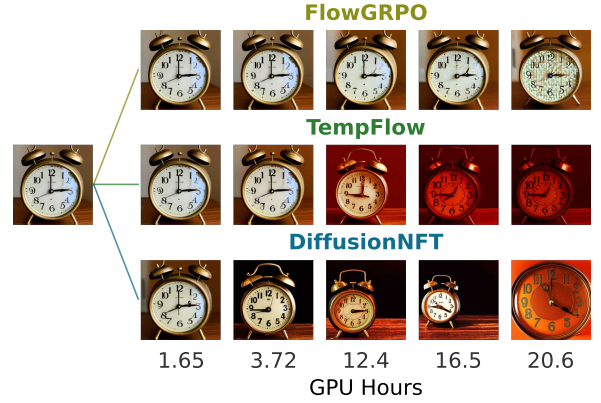


Figure 5.5: Generated images throughout training with **RL-based** fine-tuning methods. Notably, these methods require a substantial amount of training to fit the simple redness reward.

DRaFT and Direct Align lack a regularization mechanism to ensure the model stays close to the base distribution. As a result, they show a significantly faster convergence to completely red images, indicating that they severely overfit on the trained reward. Furthermore, the RL-based methods seem to take significantly longer to show any increased redness in the image, where the images from FlowGRPO stay almost the same throughout training. This slow convergence of FlowGRPO has also been reported by earlier research (He et al., 2025; Zheng et al., 2025; Choi et al., 2026).

Overall, the redness experiment shows that the methods differ substantially in how they handle an easily exploitable reward. Unregularized gradient-based methods optimize the reward quickly but exhibit severe reward hacking. Regularized gradient-based methods, especially GM, provide a better balance between reward improvement and image fidelity. RL-based methods are less prone to immediate collapse in this setting, but their optimization is considerably less sample-efficient.

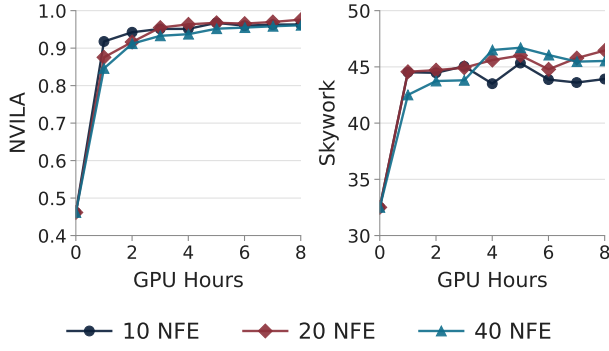


Figure 5.6: The effect of varying the number of discretization steps in the training trajectory on the convergence of **Adjoint Matching**. The standard $k = 4$ subsampling steps are used. Using fewer discretization steps has only a slight effect on early training convergence.

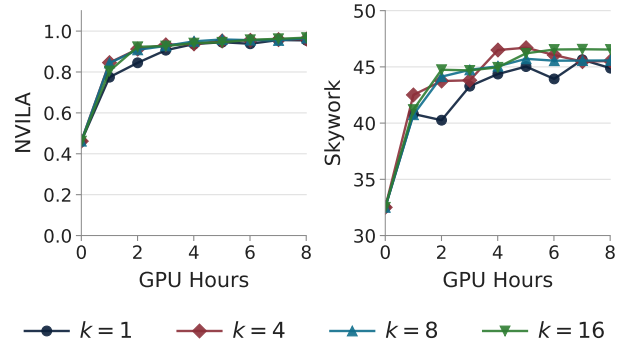


Figure 5.7: Impact of varying the number of subsampling steps, k , on the convergence of **Adjoint Matching**. The standard 40 trajectory discretization steps are used. Using fewer subsampling steps has only a minimal effect on early training convergence.

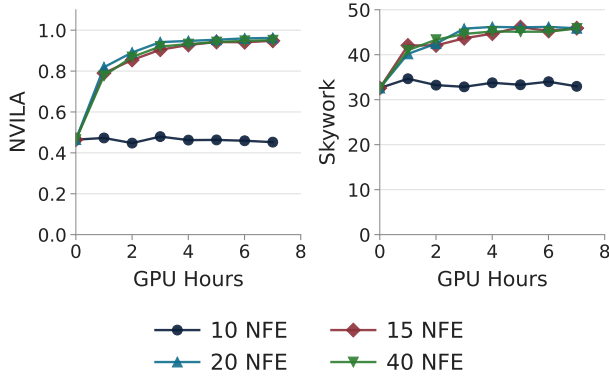


Figure 5.8: Impact of varying the number of discretization steps in the training trajectory on the convergence of **Gladjoint Matching** without subsampling of trajectory steps. Beyond 15 NFE, no significant change in convergence speed is observed.

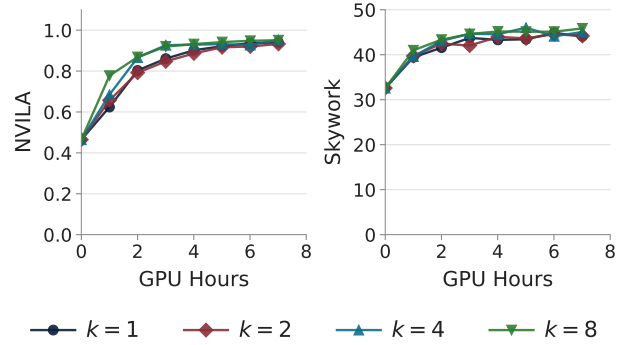


Figure 5.9: Impact of varying the number of subsampling steps, k , on the convergence of **Gladjoint Matching**. This setup uses 8 outer and 5 inner GLASS steps (40 NFE). Utilizing all trajectory steps slightly increases convergence speed.

5.2.2 NVILA

Having established the vulnerability of the baselines to extreme reward hacking and the slow convergence of RL-based methods, we next investigate reward hacking using more realistic, prompt-dependent rewards. Specifically, we apply the NVILA reward model to a subset of 10 prompts sampled from the GenEval training set. This compact dataset also enabled an efficient ablation study on the number of discretization steps in the flow match trajectory during training, which is a trade-off between generation quality and training speed. Finally, to detect potential reward hacking, we employ Skywork-VL-Reward, as preliminary results indicated that ImageReward failed to capture the reward hacking artifacts emerging from NVILA training.

Figure 5.6 shows the influence of the number of NFEs during AM training. A slight difference in initial convergence can be observed, but where all settings converges to the same Skywork and NVILA scores. Figure 5.7 shows the influence of the number of subsampling steps k on AM training. This number of subsampling steps has a slight effect on Skywork Reward, where $k = 1$ shows lower quality images in early training.

Similarly, varying the number of discretization steps in GM does not significantly impact the convergence speed, as demonstrated in Figure 5.8. Using anywhere from 15 NFE (3 outer and 5 inner steps) to 40 NFE (8 outer and 5 inner steps) during training consistently achieves a training reward of nearly 1.0. Changing the number of sampled steps in the trajectory has a greater impact on GM than on AM. As shown in Figure 5.9, using all available steps achieves faster convergence than subsampling. Employing as few as 15 NFEs, or utilizing step subsampling, yields no substantial degradation in overall performance attributable to overfitting.

These results show that both AM and GM do not overfit in this setting, as general performance measured by the Skywork reward increases during training. This effect also does not change when altering the number of discretization steps and the amount of subsampling in both fine-tuning methods.

5.3 Sparse rewards

This section addresses RQ2: *how robust is each fine-tuning method under sparse reward settings?* This setting tests whether the methods can still assign useful credit to the generation trajectory when the reward signal is limited, which is a common challenge in practical reward fine-tuning. This simulates environments such as robotics (Zhang et al., 2026; Ni et al., 2026), where an agent may only receive a reward after executing a long sequence of actions. To evaluate this, our setup utilizes a reward signal triggered only when a generated image contains a bike. Inevitably, this reward is highly sparse at the beginning of training, becoming more frequent only after the model has encountered a sufficient number of prompts specifying a 'bike'. For this purpose, we employ the Skywork reward model using a static bike-related prompt, detailed in Appendix D. Furthermore, we train exclusively on a single prompt: "A photo of a dog". This simplified setup facilitates rapid convergence and eliminates the variance introduced by diverse text conditioning. Similar to subsection 5.2.2, image quality throughout training is evaluated using ImageReward. Specifically, this metric assesses the degree to which the generated images preserve the original "dog" concept during training to incorporate the target "bike" signal. We exclude DRaFT and Direct Align from these experiments due to severe overfitting and convergence to noise. As found in section 5.2, both methods show significant reward hacking behavior.

Figure 5.10 illustrates the progression of BikeReward and ImageReward during training on a single dog-related prompt, which is also used for evaluation. Both AM and GM achieve significantly faster convergence compared to the RL-based methods. While DiffusionNFT successfully improves the targeted reward, it quickly overfits, leading to a decline in ImageReward. Figure 5.11 visualizes images generated throughout training with BikeReward. Gladjoint Matching generates slightly higher-quality images than Adjoint Matching. Specifically, Gladjoint Matching generates a distinct bike tire at the top of the image, whereas Adjoint Matching generates bike-related artifacts at the bottom.

These findings underscore the critical role of the reward gradient. Without incorporating this gradient into their objective functions, standard reinforcement learning approaches fail to learn effectively in sparse-reward environments.

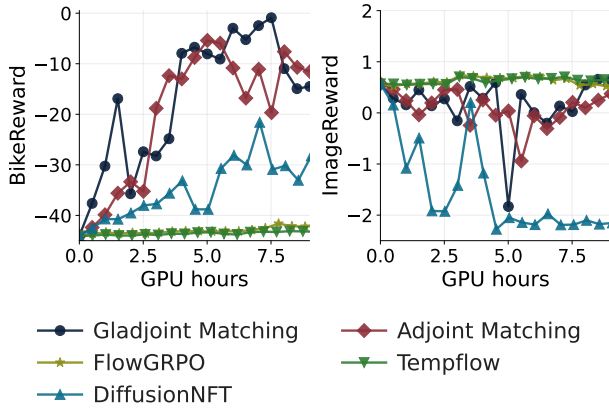


Figure 5.10: BikeReward and Skywork reward throughout training with **Gladjoint Matching**, **Adjoint Matching**, and the **RL** methods. While the RL-methods struggle with correctly fitting the target reward, both Gladjoint Matching and Adjoint Matching improve BikeReward with only a slight decrease in ImageReward.

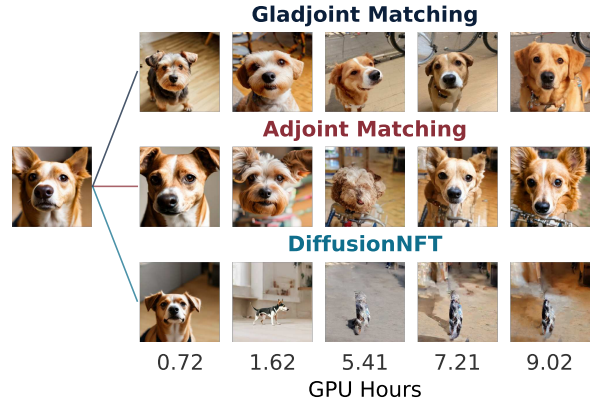


Figure 5.11: Progression of images generated throughout training with BikeReward on a single dog-related prompt. Gladjoint Matching produces the highest quality images, followed by Adjoint Matching, whereas DiffusionNFT yields noisy results.

5.4 General Performance

This section addresses RQ3: *how effectively does each fine-tuning method improve the performance of the base model?* After evaluating reward hacking and sparse rewards in controlled settings, we now test whether earlier findings extend to general text-to-image generation performance. GenEval measures compositional prompt alignment, while UniGenBench++ evaluates a broader set of visual and semantic capabilities. To assess the efficiency and effectiveness of each fine-tuning method, their performance is evaluated given a fixed training budget of 200 GPU hours.

Quantitative Performance. Figure 5.12 illustrates the GenEval benchmark performance and EMA-averaged NVILA reward progression throughout training. Notably, DiffusionNFT obtains a significantly lower reward, which is attributable to using only 10 trajectory discretization steps and omitting CFG. Furthermore, DiffusionNFT shows reward hacking behavior, resulting in a drop in GenEval scores during training. In contrast, this reward-hacking behavior is not observed on UniGenBench++ when training on the large combined dataset from Eyring et al. (2025), as shown in Figure 5.13. In this setting, DRaFT converges the fastest, followed by Adjoint Matching and then our Gladjoint Matching method. Across both Figure 5.12 and Figure 5.13, Direct Align training displays a general instability: both the training reward and benchmark performance oscillate substantially throughout training.

Table 5.1 and Table 5.2 show the final GenEval and UniGenBench++ benchmark scores for the tested fine-tuning methods. DRaFT slightly outperforms Direct Align, Adjoint Matching, and our proposed Gladjoint Matching in both NVILA rewards and final benchmark scores. On GenEval, Gladjoint Matching is the second-best method overall and substantially outperforms the RL-based baselines. On UniGenBench++, Adjoint Matching ranks second overall, followed by Gladjoint Matching. These results suggest that DRaFT benefits from its simplified optimization scheme, which omits the regularization terms that typically constrain and slow the learning process. Notably, while we previously demonstrated that DRaFT and Direct Align are prone to significant reward hacking in section 5.2, this behavior does not result in reduced generalization in the current setting. This stability can likely be attributed to the current

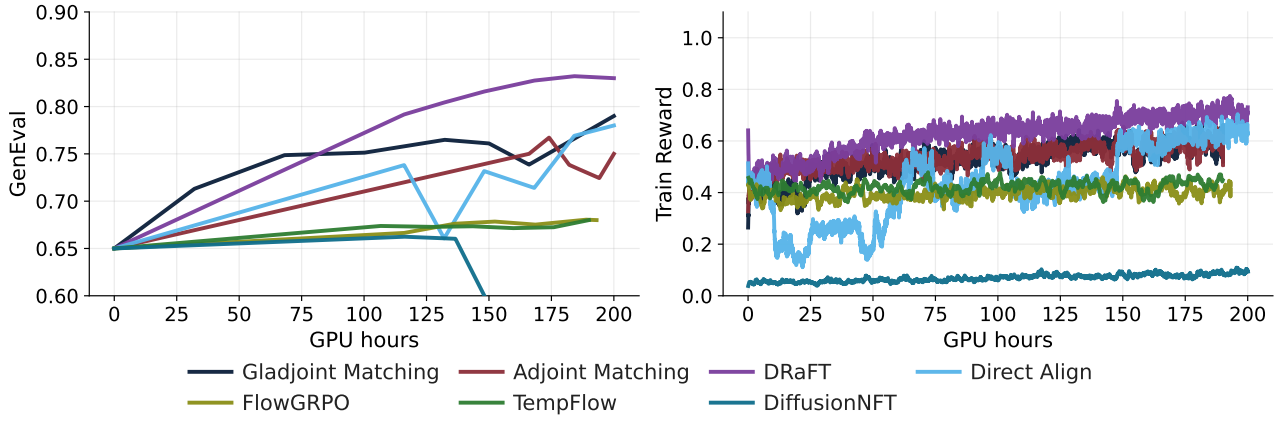


Figure 5.12: GenEval benchmark performance and the EMA-averaged NVILA reward throughout training. DRaFT converges fastest, followed by Gladjoint Matching, Direct Align, and Adjoint Matching. In contrast, DiffusionNFT significantly overfits, resulting in decreased GenEval Performance.

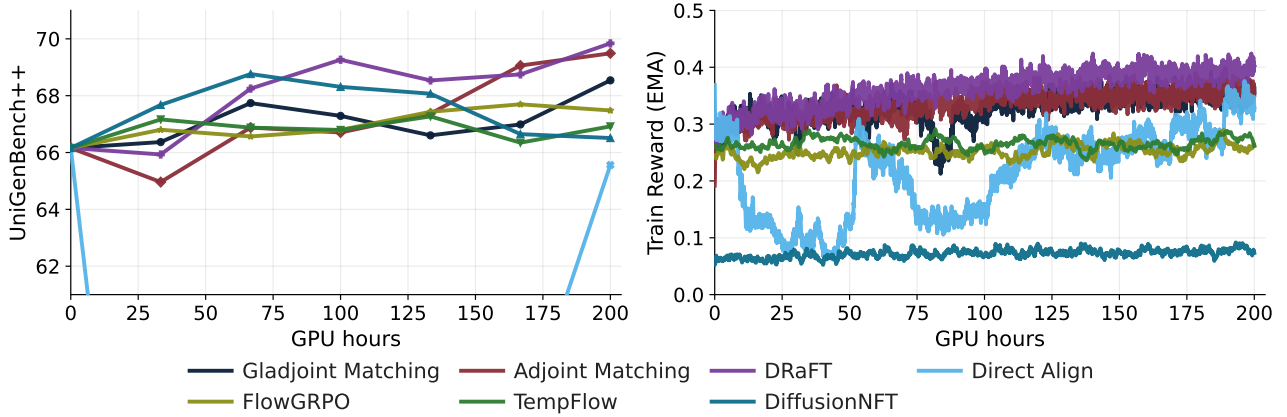


Figure 5.13: UniGenBench++ performance and EMA-averaged NVILA reward throughout training. DRaFT converges fastest, followed by Adjoint Matching, Gladjoint Matching, and the RL methods. During Direct Align training, both benchmark performance and reward oscillate substantially, indicating training instability.

Model	Overall	Single Obj.	Two Obj.	Counting	Colors	Position	Attr. Binding
SD3.5-M	0.65	0.98	0.80	0.57	0.82	0.23	0.53
<i>RL methods</i>							
FlowGRPO	0.68	0.98	0.82	0.63	0.81	0.27	0.58
TempFlow	0.68	0.98	0.83	0.61	0.82	0.27	0.58
DiffusionNFT	0.46	0.81	0.47	0.28	0.69	0.18	0.30
<i>Gradient-based methods</i>							
DRaFT	0.83	1.00	0.94	0.82	0.90	0.49	0.86
Direct Align	0.78	0.98	0.90	0.73	0.88	<u>0.43</u>	<u>0.79</u>
Adjoint Matching	0.75	<u>0.99</u>	0.88	0.69	0.88	0.33	0.71
Gladjoint Matching	<u>0.79</u>	0.98	<u>0.91</u>	<u>0.78</u>	<u>0.90</u>	0.42	0.76

Table 5.1: Benchmarking Results of the fine-tuning methods on GENEVAL. Best scores are in **bold**, second-best in underlined. DRaFT achieves the highest final performance, followed by Gladjoint Matching, significantly outperforming the RL-based approaches. In contrast, DiffusionNFT exhibits degraded performance.

Method	Overall	Style	World Know.	Attribute	Action	Relation.	Logic.Reason.	Grammar	Compound	Layout	Text
SD3.5M	66.16	92.50	88.77	69.55	62.26	72.21	33.72	65.91	59.54	71.64	15.80
<i>RL Methods</i>											
FlowGRPO	67.48	90.80	89.56	71.37	65.59	71.83	36.24	<u>66.71</u>	62.11	74.25	16.67
TempFlow	66.92	<u>92.00</u>	89.87	69.66	65.11	70.81	34.86	66.31	59.41	72.57	15.52
DiffusionNFT	66.51	79.30	87.18	68.80	67.87	73.60	38.53	68.05	<u>65.72</u>	75.37	<u>20.40</u>
<i>Gradient-based methods</i>											
DRaFT	69.84	85.60	<u>92.41</u>	77.35	69.39	77.28	44.95	64.04	68.94	<u>75.75</u>	16.09
Direct Align	65.56	88.90	84.65	69.55	64.16	71.32	41.06	64.30	63.53	71.83	7.47
Adjoint Matching	<u>69.49</u>	90.50	92.56	71.69	<u>72.34</u>	73.98	39.68	64.04	63.53	75.93	20.40
Gladjoint Matching	68.54	87.70	92.25	<u>72.76</u>	73.29	<u>74.87</u>	<u>43.35</u>	64.30	64.18	71.83	14.08

Table 5.2: Benchmarking Results of the fine-tuning methods on UNIGENBENCH++. Best scores are in **bold**, second-best in underlined. DRaFT achieves the highest final performance, followed by Adjoint Matching and Gladjoint Matching. In contrast, Direct Align shows degraded performance.

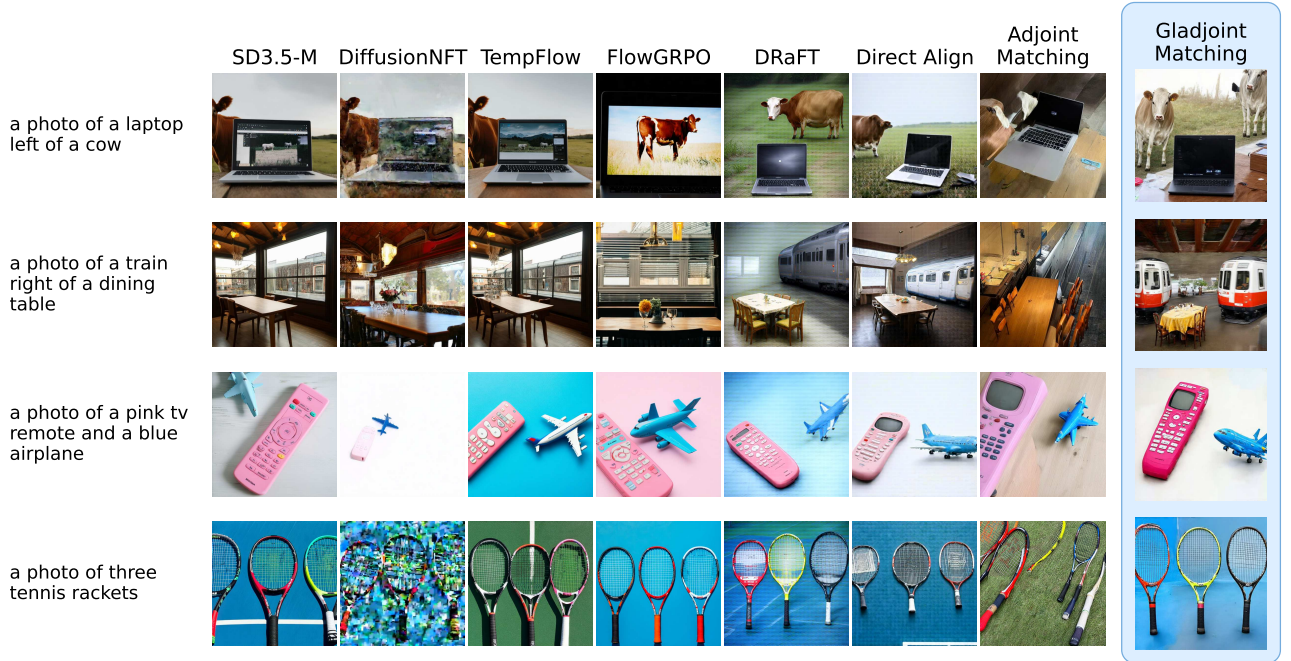


Figure 5.14: Qualitative comparison of images generated by different fine-tuning methods against the baseline on **GenEval** test prompts. Gladjoint Matching follows the prompts well but exhibits cheating behavior on position-based prompts. DRaFT and Direct Align show grid-like artifacts, suggesting reward-hacking behavior.

experimental setup: the NVILA reward model presents a significantly more complex objective to exploit, a difficulty further compounded by the robust scale of the 50,000-entry training dataset. Nevertheless, with additional training, final GenEval performance may eventually decrease as more severe reward hacking behavior begins to emerge.

Qualitative Performance. The qualitative results in Figure 5.14 further demonstrate the improved performance of Gladjoint Matching compared with Adjoint Matching. Our method enhances both text-image alignment and overall image quality. However, we observe a minor form of reward hacking, where similar objects are generated on both the left and right sides of the foreground object. This allows the generated images to satisfy positional constraints of the GenEval benchmark without fully capturing the underlying spatial relation. This behavior may also partly explain the substantial increase in position score achieved by Gladjoint Matching,



Figure 5.15: Qualitative comparison of images generated by the fine-tuning methods trained on the large combined dataset from Eyring et al. (2025), compared with the baseline on **UniGen-Bench++** test prompts. Gladjoint Matching and DRaFT exhibit similar reward alignment performance, both significantly improving over the baseline. The grid-line artifacts observed for DRaFT and Direct Align persist on these prompts.

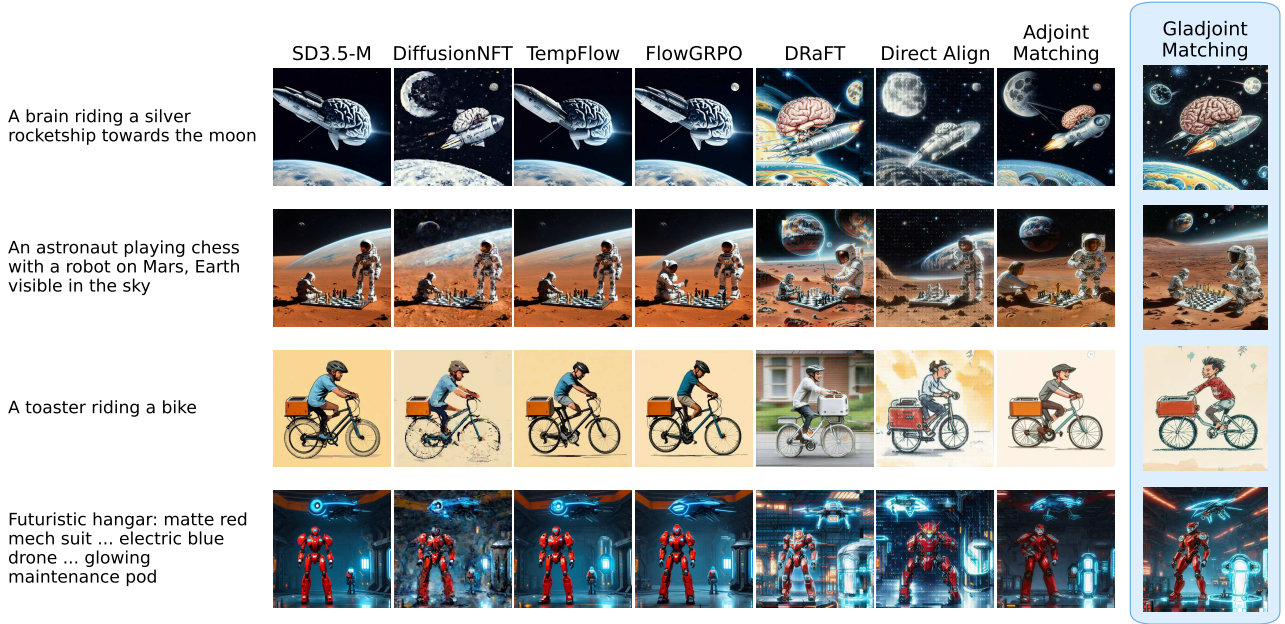


Figure 5.16: Qualitative comparison of images generated by the baseline model and by models fine-tuned with the different methods on the large combined training set from Eyring et al. (2025), evaluated on **custom prompts**. Gladjoint Matching, Adjoint Matching, and DRaFT achieve comparable reward alignment. DRaFT and Direct Align again show grid-line artifacts.

compared with Adjoint matching on this specific benchmark.

The figure also supports our previous findings: DiffusionNFT exhibits reward hacking by introducing noise into the generated images, while DRaFT and Direct Align show reward hacking behavior through grid-line artifacts, as also reported by Clark et al. (2024). These artifacts suggest that strong quantitative performance does not necessarily imply robust qualitative align-

ment. Figure 5.15 and Figure 5.16 further compare images generated by the baseline model with those produced by models fine-tuned using different methods on the large combined training set from Eyring et al. (2025), using UniGenBench++ test prompts and custom prompts from Holderrieth et al. (2026a). These results again reveal the reward-hacking artifacts induced by DRaFT and Direct Align training. Across these prompts, Gladjoint Matching, Adjoint Matching, and DRaFT exhibit comparable reward-alignment performance, with the best-performing method varying across individual samples.

Overall, these results show that the gradient-based fine-tuning methods provide the strongest improvements in general text-to-image performance under a fixed training budget. DRaFT achieves the highest quantitative scores, but its qualitative artifacts indicate reward hacking behavior. In contrast, Gladjoint Matching offers a more balanced trade-off, improving over the baseline and Adjoint Matching.

Chapter 6

Conclusion

In this work, we explored reward-based fine-tuning approaches for flow matching models, aiming to efficiently align generated images with downstream tasks and complex human preferences. To establish a clear baseline and address a critical gap in current literature, we conducted a comprehensive comparative analysis of existing RL and gradient-based fine-tuning methods. Guided by these insights, we introduce an improved gradient-based optimization method by incorporating efficient GLASS trajectories and adapted regularization strategies within the Adjoint Matching framework.

First, we introduced **Gladjoint Matching**, a novel approach that integrates the memory-less GLASS transition sampler with the Adjoint Matching objective. Our theoretical findings show that this approach preserves the endpoint independence strictly required for unbiased Adjoint Matching. Next, we explored the occurrence and mitigation of reward hacking by systematically evaluating the models under both simple visual heuristics and prompt-dependent reward functions. The results show that by utilizing the GLASS trajectories during training, Gladjoint Matching reduces the amount of overfitting that occurred in other fine-tuning methods while increasing convergence speed. Building on these findings, we introduced a sparse-feedback environment by implementing a VLM reward function that evaluates images for the occurrence of bicycles. Experiments show that Gladjoint Matching maintains remarkable robustness in these settings, quickly converging to the target distribution and generating higher-quality images, whereas baseline methods struggle with credit assignment and often collapse into noise or not improve the trained reward. Finally, we evaluated the broader generalizability of our method across the comprehensive GenEval and UniGenBench++ benchmarks. This evaluation demonstrates that Gladjoint Matching effectively translates its theoretical and training advantages into superior prompt alignment, spatial positioning, and overall compositional accuracy, surpassing current state-of-the-art fine-tuning methods within a fixed computational budget.

Limitations and Future Work

While our work demonstrates that utilizing GLASS trajectories to fine-tune flow matching models improves training efficiency, several limitations remain, offering multiple directions for future research. First, although GLASS trajectories enhance efficiency, they do not address the fundamental requirement for stochastic trajectories during gradient-based training. Efficiency could be further optimized by adapting techniques like Adjoint Matching to utilize ODE transitions. Second, our method leverages flow matching, which offers superior sampling efficiency compared to the diffusion paradigm. However, our method could be further enhanced by incorporating flow maps (Boffi et al., 2025) which would further improve efficiency. Third, process rewards, which have recently shown promise in VLM and LLM research (Khalifa et al., 2025; Wang et al., 2025a) may offer a useful direction for fine-tuning image generation models in settings where final outcome rewards are sparse. Adapting such ideas to generative image models

could provide denser intermediate supervision, although defining meaningful intermediate rewards for the image generation process remains an open challenge (Xu et al.; Yang et al., 2024). Finally, our current analysis is restricted to a single model architecture (SD-3.5M). Extending these experiments across a broader variety of model types represents an important direction for future research to validate the generalizability of our findings.

Bibliography

- M. S. Albergo and E. Vanden-Eijnden. BUILDING NORMALIZING FLOWS WITH STOCHASTIC INTERPOLANTS. 2023.
- M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein GAN, Jan. 2017. URL <https://arxiv.org/abs/1701.07875v3>.
- R. E. Bellman and S. E. Dreyfus. Applied Dynamic Programming. pages 1–390, 2015. URL <https://www.torrossa.com/en/resources/an/5559786>.
- N. M. Boffi, M. S. Albergo, and E. Vanden-Eijnden. Flow map matching with stochastic interpolants: A mathematical framework for consistency models, June 2025. URL <http://arxiv.org/abs/2406.07507>. arXiv:2406.07507 [cs].
- G. Bredell, K. Flouris, K. Chaitanya, E. Erdil, and E. Konukoglu. Explicitly Minimizing the Blur Error of Variational Autoencoders, Apr. 2023. URL <https://arxiv.org/abs/2304.05939v1>.
- R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural Ordinary Differential Equations. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/hash/69386f6bb1dfed68692a24c8686939b9-Abstract.html.
- B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R. Girdhar. Masked-Attention Mask Transformer for Universal Image Segmentation. pages 1290–1299, 2022. URL https://openaccess.thecvf.com/content/CVPR2022/html/Cheng_Masked-Attention_Mask_Transformer_for_Universal_Image_Segmentation_CVPR_2022_paper.html.
- J. Choi, Y. Zhu, W. Guo, P. Molodyk, B. Yuan, J. Bai, Y. Xin, M. Tao, and Y. Chen. Rethinking the Design Space of Reinforcement Learning for Diffusion Models: On the Importance of Likelihood Estimation Beyond Loss Design, Feb. 2026. URL <http://arxiv.org/abs/2602.04663>. arXiv:2602.04663 [cs].
- P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences, June 2017. URL <https://arxiv.org/abs/1706.03741v4>.
- K. Clark, P. Vicol, K. Swersky, and D. J. Fleet. Directly Fine-Tuning Diffusion Models on Differentiable Rewards, June 2024. URL <http://arxiv.org/abs/2309.17400>. arXiv:2309.17400 [cs].
- DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan,

- J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *Nature*, 645(8081): 633–638, Sept. 2025. ISSN 0028-0836, 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://arxiv.org/abs/2501.12948>. arXiv:2501.12948 [cs.CL].
- P. Dhariwal and A. Nichol. Diffusion Models Beat GANs on Image Synthesis. In *Advances in Neural Information Processing Systems*, volume 34, pages 8780–8794. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/49ad23d1ec9fa4bd8d77d02681df5cfa-Abstract.html.
- M. Ding, Z. Yang, W. Hong, W. Zheng, C. Zhou, D. Yin, J. Lin, X. Zou, Z. Shao, H. Yang, and J. Tang. CogView: Mastering Text-to-Image Generation via Transformers. In *Advances in Neural Information Processing Systems*, volume 34, pages 19822–19835. Curran Associates, Inc., 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/a4d92e2cd541fca87e4620aba658316d-Abstract.html>.
- C. Domingo-Enrich, M. Drozdal, B. Karrer, and R. T. Q. Chen. Adjoint Matching: Fine-tuning Flow and Diffusion Generative Models with Memoryless Stochastic Optimal Control, Jan. 2025. URL <http://arxiv.org/abs/2409.08861>. arXiv:2409.08861 [cs].
- P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, K. Lacey, A. Goodwin, Y. Marek, and R. Rombach. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis, Mar. 2024. URL <http://arxiv.org/abs/2403.03206>. arXiv:2403.03206 [cs].
- L. Eyring, S. Karthik, A. Dosovitskiy, N. Ruiz, and Z. Akata. Noise Hypernetworks: Amortizing Test-Time Compute in Diffusion Models, Aug. 2025. URL <http://arxiv.org/abs/2508.09968>. arXiv:2508.09968 [cs].
- W. Feng, X. He, T.-J. Fu, V. Jampani, A. R. Akula, P. Narayana, S. Basu, X. E. Wang, and W. Y. Wang. Training-Free Structured Diffusion Guidance for Compositional Text-to-Image Synthesis. Sept. 2022. URL <https://openreview.net/forum?id=PUIqjT4rzq7>.
- D. Ghosh, H. Hajishirzi, and L. Schmidt. GenEval: An object-focused framework for evaluating text-to-image alignment. *Advances in Neural Information Processing Systems*, 36:52132–52152, Dec. 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/a3bf71c7c63f0c3bcb7ff67c67b1e7b1-Abstract-Datasets_and_Benchmarks.html.

- I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative Adversarial Nets. In *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014. URL <https://proceedings.neurips.cc/paper/2014/hash/f033ed80deb0234979a61f95710dbe25-Abstract.html>.
- X. He, S. Fu, Y. Zhao, W. Li, J. Yang, D. Yin, F. Rao, and B. Zhang. TempFlow-GRPO: When Timing Matters for GRPO in Flow Models, Oct. 2025. URL <http://arxiv.org/abs/2508.04324>. arXiv:2508.04324 [cs].
- J. Ho and T. Salimans. Classifier-Free Diffusion Guidance, July 2022. URL <http://arxiv.org/abs/2207.12598>. arXiv:2207.12598 [cs].
- J. Ho, A. Jain, and P. Abbeel. Denoising Diffusion Probabilistic Models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>.
- P. Holderrieth, D. Chen, L. Eyring, I. Shah, G. Anantharaman, Y. He, Z. Akata, T. Jaakkola, N. M. Boffi, and M. Simchowitz. Diamond Maps: Efficient Reward Alignment via Stochastic Flow Maps, May 2026a. URL <http://arxiv.org/abs/2602.05993>. arXiv:2602.05993 [cs.LG].
- P. Holderrieth, U. Singer, T. Jaakkola, R. T. Q. Chen, Y. Lipman, and B. Karrer. GLASS Flows: Transition Sampling for Alignment of Flow and Diffusion Models, Feb. 2026b. URL <http://arxiv.org/abs/2509.25170>. arXiv:2509.25170 [cs].
- X. Hou, L. Shen, K. Sun, and G. Qiu. Deep Feature Consistent Variational Autoencoder. In *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, pages 1133–1141, Mar. 2017. doi: 10.1109/WACV.2017.131. URL <https://ieeexplore.ieee.org/abstract/document/7926714>.
- N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly. Parameter-Efficient Transfer Learning for NLP, Feb. 2019. URL <https://arxiv.org/abs/1902.00751v2>.
- E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models, Oct. 2021. URL <http://arxiv.org/abs/2106.09685>. arXiv:2106.09685 [cs.CL].
- G. Huang, Y. Li, G. Pleiss, Z. Liu, J. E. Hopcroft, and K. Q. Weinberger. Snapshot Ensembles: Train 1, get M for free, Apr. 2017. URL <http://arxiv.org/abs/1704.00109>. arXiv:1704.00109 [cs.LG].
- K. Huang, K. Sun, E. Xie, Z. Li, and X. Liu. T2I-CompBench: A Comprehensive Benchmark for Open-world Compositional Text-to-image Generation. *Advances in Neural Information Processing Systems*, 36:78723–78747, Dec. 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/f8ad010cdd9143dbb0e9308c093aff24-Abstract-Datasets_and_Benchmarks.html.
- M. Khalifa, R. Agarwal, L. Logeswaran, J. Kim, H. Peng, M. Lee, H. Lee, and L. Wang. Process Reward Models That Think, Dec. 2025. URL <http://arxiv.org/abs/2504.16828>. arXiv:2504.16828 [cs.LG].
- D. P. Kingma and M. Welling. Auto-Encoding Variational Bayes, Dec. 2022. URL <http://arxiv.org/abs/1312.6114>. arXiv:1312.6114 [stat.ML].

- Y. Kirstain, A. Polyak, U. Singer, S. Matiana, J. Penna, and O. Levy. Pick-a-Pic: An Open Dataset of User Preferences for Text-to-Image Generation. *Advances in Neural Information Processing Systems*, 36:36652–36663, Dec. 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/73aacd8b3b05b4b503d58310b523553c-Abstract-Conference.html.
- S. Kullback and R. A. Leibler. On Information and Sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. ISSN 0003-4851. URL <https://www.jstor.org/stable/2236703>.
- K. Lee, H. Liu, M. Ryu, O. Watkins, Y. Du, C. Boutilier, P. Abbeel, M. Ghavamzadeh, and S. S. Gu. Aligning Text-to-Image Models using Human Feedback, Feb. 2023a. URL <https://arxiv.org/abs/2302.12192v1>.
- S. Lee, B. Kim, and J. C. Ye. Minimizing Trajectory Curvature of ODE-based Generative Models. In *Proceedings of the 40th International Conference on Machine Learning*, pages 18957–18973. PMLR, July 2023b. URL <https://proceedings.mlr.press/v202/lee23j.html>.
- J. Li, D. Li, C. Xiong, and S. Hoi. BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation. In *Proceedings of the 39th International Conference on Machine Learning*, pages 12888–12900. PMLR, June 2022. URL <https://proceedings.mlr.press/v162/li22n.html>.
- Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow Matching for Generative Modeling, Feb. 2023. URL <http://arxiv.org/abs/2210.02747>. arXiv:2210.02747 [cs].
- J. Liu, G. Liu, J. Liang, Y. Li, J. Liu, X. Wang, P. Wan, D. Zhang, and W. Ouyang. Flow-GRPO: Training Flow Matching Models via Online RL, Oct. 2025. URL <http://arxiv.org/abs/2505.05470>. arXiv:2505.05470 [cs].
- X. Liu, C. Gong, and Q. Liu. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow, Sept. 2022. URL <http://arxiv.org/abs/2209.03003>. arXiv:2209.03003 [cs].
- Z. Liu, L. Zhu, B. Shi, Z. Zhang, Y. Lou, S. Yang, H. Xi, S. Cao, Y. Gu, D. Li, X. Li, Y. Fang, Y. Chen, C.-Y. Hsieh, D.-A. Huang, A.-C. Cheng, V. Nath, J. Hu, S. Liu, R. Krishna, D. Xu, X. Wang, P. Molchanov, J. Kautz, H. Yin, S. Han, and Y. Lu. NVILA: Efficient Frontier Visual Language Models, Dec. 2024. URL <https://arxiv.org/abs/2412.04468v3>.
- I. Loshchilov and F. Hutter. Decoupled Weight Decay Regularization, Jan. 2019. URL <http://arxiv.org/abs/1711.05101>. arXiv:1711.05101 [cs.LG].
- N. Ma, M. Goldstein, M. S. Albergo, N. M. Boffi, E. Vanden-Eijnden, and S. Xie. SiT: Exploring Flow and Diffusion-based Generative Models with Scalable Interpolant Transformers, Sept. 2024. URL <http://arxiv.org/abs/2401.08740>. arXiv:2401.08740 [cs.CV].
- T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida. Spectral Normalization for Generative Adversarial Networks, Feb. 2018. URL <http://arxiv.org/abs/1802.05957>. arXiv:1802.05957 [cs.LG].
- C. Mou, Z. Zhuang, X. Chen, and Y. Zhang. Beyond Uniform Credit Assignment: Selective Eligibility Traces for RLVR, May 2026. URL <https://arxiv.org/abs/2605.05965v1>.

- Z. Ni, Y. Li, R. Jiao, S. S. Zhan, S. Chen, Z. Yin, M. Chen, P. Torr, Z. Wang, and Q. Zhu. CreFlow: Corrective Reflow for Sparse-Reward Embodied Video Diffusion RL, May 2026. URL <https://arxiv.org/abs/2605.14274v1>.
- P. Parthasarathi, M. Reymond, B. Chen, Y. Cui, and S. Chandar. GRPO-\$\$: Credit Assignment improves LLM Reasoning, 2025. URL <https://arxiv.org/abs/2510.00194>. Version Number: 1.
- L. S. Pontryagin. *Mathematical Theory of Optimal Processes*. CRC Press, Mar. 1987. ISBN 978-2-88124-077-5. Google-Books-ID: kwzq0F4cBVAC.
- A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning Transferable Visual Models From Natural Language Supervision, Feb. 2021. URL <https://arxiv.org/abs/2103.00020v1>.
- R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-Resolution Image Synthesis with Latent Diffusion Models, Dec. 2021. URL <https://arxiv.org/abs/2112.10752v2>.
- N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman. DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22500–22510, Vancouver, BC, Canada, June 2023. IEEE. ISBN 979-8-3503-0129-8. doi: 10.1109/CVPR52729.2023.02155. URL <https://ieeexplore.ieee.org/document/10204880/>.
- C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E. L. Denton, K. Ghasemipour, R. Gontijo Lopes, B. Karagol Ayan, T. Salimans, J. Ho, D. Fleet, and M. Norouzi. Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding. *Advances in Neural Information Processing Systems*, 35:36479–36494, Dec. 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/ec795aeadae0b7d230fa35cbaf04c041-Abstract-Conference.html.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms, Aug. 2017. URL <http://arxiv.org/abs/1707.06347>. arXiv:1707.06347 [cs.LG].
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, Apr. 2024. URL <http://arxiv.org/abs/2402.03300>. arXiv:2402.03300 [cs].
- X. Shen, Z. Li, Z. Yang, S. Zhang, Y. Zhang, D. Li, C. Wang, Q. Lu, and Y. Tang. Directly Aligning the Full Diffusion Trajectory with Fine-Grained Human Preference, 2025. URL <https://arxiv.org/abs/2509.06942>. Version Number: 3.
- J. Sheng, H. Zhao, H. Chen, D. D. Yao, and W. Tang. Understanding Sampler Stochasticity in Training Diffusion Models for RLHF, Dec. 2025. URL <http://arxiv.org/abs/2510.10767>. arXiv:2510.10767 [cs.LG].
- Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-Based Generative Modeling through Stochastic Differential Equations, Feb. 2021. URL <http://arxiv.org/abs/2011.13456>. arXiv:2011.13456 [cs].

- R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy Gradient Methods for Reinforcement Learning with Function Approximation. In *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html.
- H. Tran, Z. Yao, and H. Yu. Exploiting Tree Structure for Credit Assignment in RL Training of LLMs, Sept. 2025. URL <https://arxiv.org/abs/2509.18314v2>.
- P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, Y. Fan, K. Dang, M. Du, X. Ren, R. Men, D. Liu, C. Zhou, J. Zhou, and J. Lin. Qwen2-VL: Enhancing Vision-Language Model’s Perception of the World at Any Resolution, Sept. 2024. URL <https://arxiv.org/abs/2409.12191v2>.
- W. Wang, Z. Gao, L. Chen, Z. Chen, J. Zhu, X. Zhao, Y. Liu, Y. Cao, S. Ye, X. Zhu, L. Lu, H. Duan, Y. Qiao, J. Dai, and W. Wang. VisualPRM: An Effective Process Reward Model for Multimodal Reasoning, Mar. 2025a. URL <http://arxiv.org/abs/2503.10291>. arXiv:2503.10291 [cs.CV].
- X. Wang, P. Wang, J. Pei, W. Shen, Y. Peng, Y. Hao, W. Qiu, A. Jian, T. Xie, X. Song, Y. Liu, and Y. Zhou. Skywork-VL Reward: An Effective Reward Model for Multimodal Understanding and Reasoning, June 2025b. URL <http://arxiv.org/abs/2505.07263>. arXiv:2505.07263 [cs].
- Y. Wang, Z. Li, Y. Zang, J. Bu, Y. Zhou, Y. Xin, J. He, C. Wang, Q. Lu, C. Jin, and J. Wang. UniGenBench++: A Unified Semantic Evaluation Benchmark for Text-to-Image Generation, Feb. 2026. URL <http://arxiv.org/abs/2510.18701>. arXiv:2510.18701 [cs].
- R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, May 1992. ISSN 1573-0565. doi: 10.1007/BF00992696. URL <https://doi.org/10.1007/BF00992696>.
- J. Xu, X. Liu, Y. Wu, Y. Tong, Q. Li, M. Ding, J. Tang, and Y. Dong. ImageReward: Learning and Evaluating Human Preferences for Text-to-Image Generation.
- Z. Xue, J. Wu, Y. Gao, F. Kong, L. Zhu, M. Chen, Z. Liu, W. Liu, Q. Guo, W. Huang, and P. Luo. DanceGRPO: Unleashing GRPO on Visual Generation, Aug. 2025. URL <http://arxiv.org/abs/2505.07818>. arXiv:2505.07818 [cs].
- S. Yang, T. Chen, and M. Zhou. A Dense Reward View on Aligning Text-to-Image Diffusion with Preference, May 2024. URL <http://arxiv.org/abs/2402.08265>. arXiv:2402.08265 [cs.CV].
- T. Zhang, C. Yu, S. Su, and Y. Wang. ReinFlow: Fine-tuning Flow Matching Policy with Online Reinforcement Learning, Jan. 2026. URL <http://arxiv.org/abs/2505.22094>. arXiv:2505.22094 [cs].
- K. Zheng, H. Chen, H. Ye, H. Wang, Q. Zhang, K. Jiang, H. Su, S. Ermon, J. Zhu, and M.-Y. Liu. DiffusionNFT: Online Diffusion Reinforcement with Forward Process, Sept. 2025. URL <http://arxiv.org/abs/2509.16117>. arXiv:2509.16117 [cs].

Appendix

A Improved regularization

Figure A.1 shows the difference in reward hacking when utilizing our changed regularization in Adjoint Matching fine-tuning. When utilizing similar amounts of regularization, our $\lambda_{reg} = 0.3$ shows significantly less reward hacking than the original $\lambda_{grad} = 1000$. Furthermore, $\lambda_{reg} = 0.5$ improves training convergence while decreasing the amount of reward hacking compared to $\lambda_{grad} = 50$.

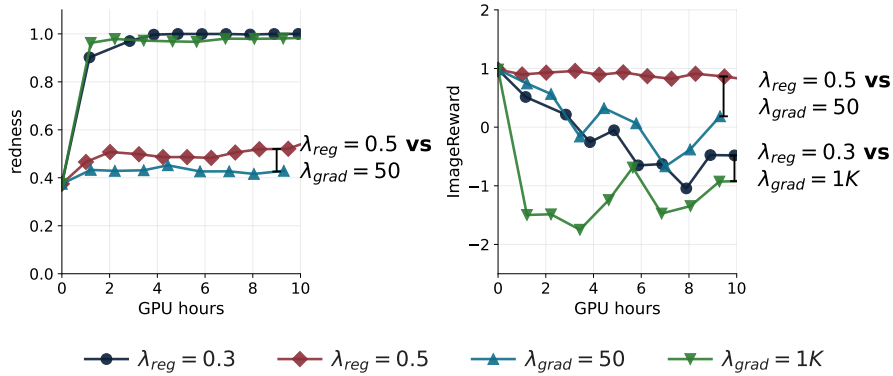


Figure A.1: Difference in performance when utilizing a downscaling of the regularization term (λ_{reg}) instead of the original upscaling of the reward gradient (λ_{grad}).

B Hyperparameters

Gradient-based methods. We use DRaFT-8 for all experiments, propagating the reward through the final eight steps of each trajectory. In subsection 5.2.2, section 5.3, and section 5.4, both Gladjoint Matching and Adjoint Matching use $\lambda=0.005$ to ensure a fair comparison. In section 5.3 and section 5.4, Adjoint Matching uses the standard configuration of 40 discretization steps with 4 subsampling steps, whereas Gladjoint Matching uses 4 outer and 5 inner GLASS discretization steps without subsampling.

RL-based methods. Based on the experiments in Appendix C, we find that $\beta=0.04$ for FlowGRPO and DiffusionNFT, and $\beta=0.06$ for TempFlow, offer the best trade-off between increasing the trained reward and staying close to the base model. We use these values for all experiments.

C Redness ALL Hyperparameters

Below, in Figure C.1, Figure C.2, Figure C.3, Figure C.4, Figure C.5, and Figure C.6, the results for training on the redness reward are shown. The figures show the influence of the regularization on the amount of reward hacking and speed of convergence. Furthermore, we perform an ablation on the removal of CFG during training for the FlowGRPO and Tempflow methods in Figure C.3 and Figure C.5. They both show increased convergence when not utilizing CFG during training.

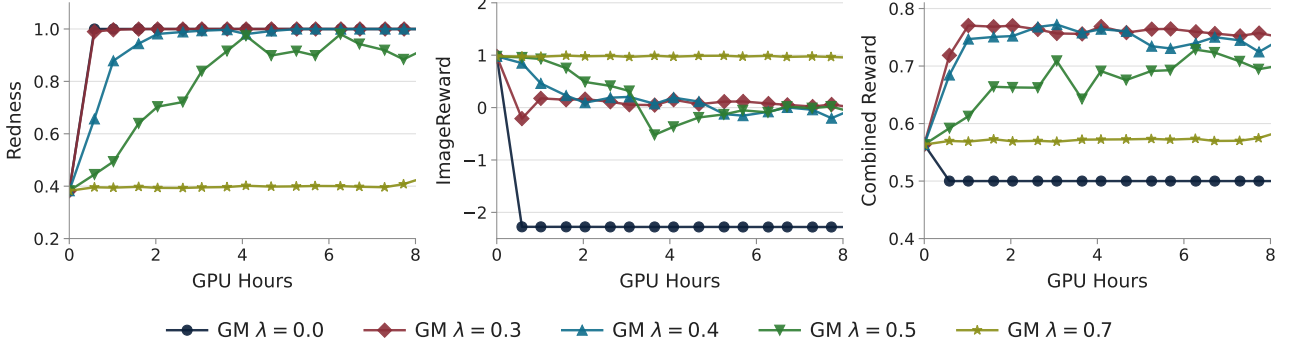


Figure C.1: Influence of different regularization on the training of Gladjoint Matching

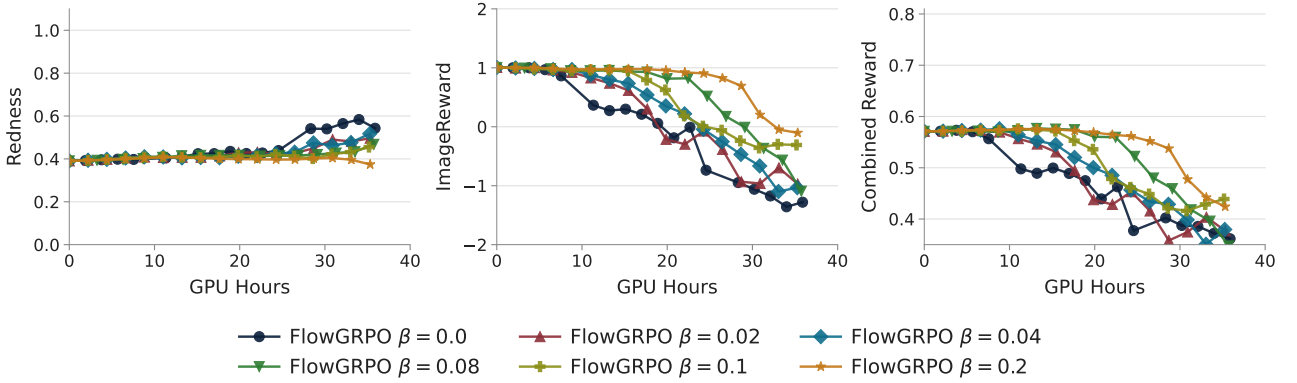


Figure C.2: Influence of different regularization on the training of FlowGRPO when utilizing CFG.

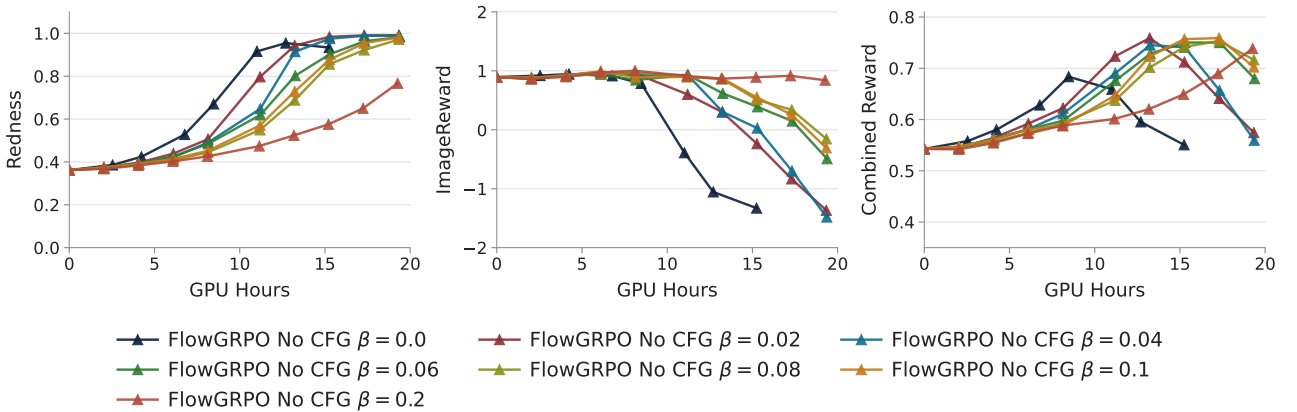


Figure C.3: Influence of different regularization on the training of FlowGRPO when not utilizing CFG.

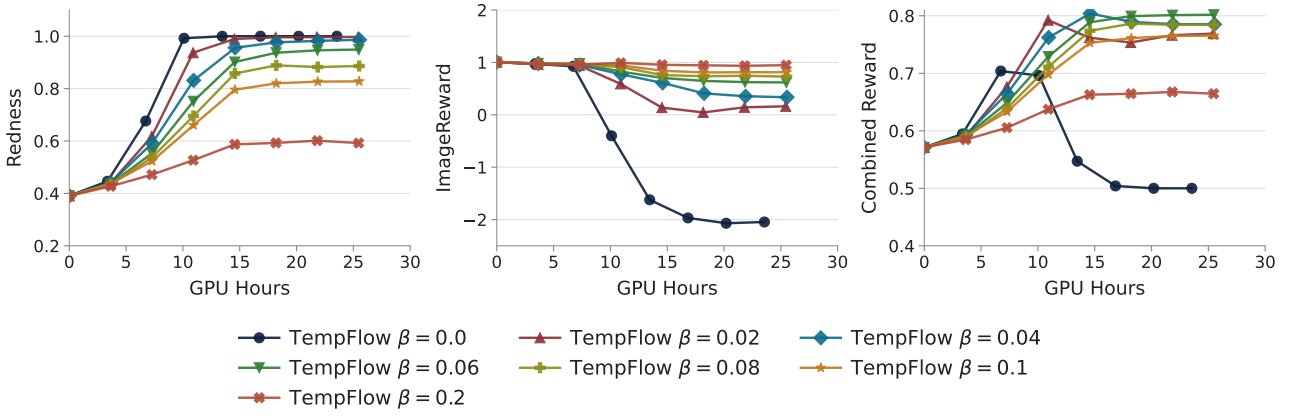


Figure C.4: Influence of different regularization on the training of TempFlow when utilizing CFG.

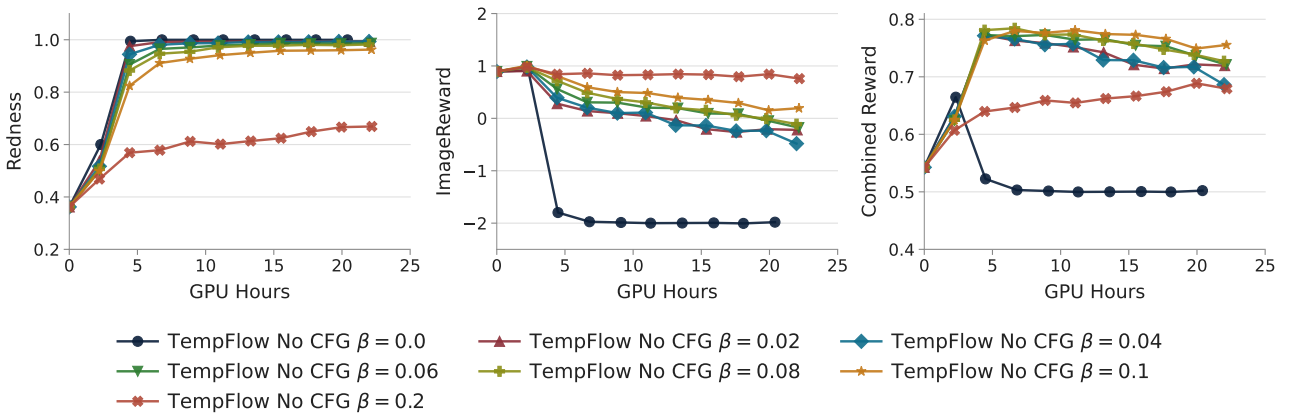


Figure C.5: Influence of different regularization on the training of TempFlow when not utilizing CFG.

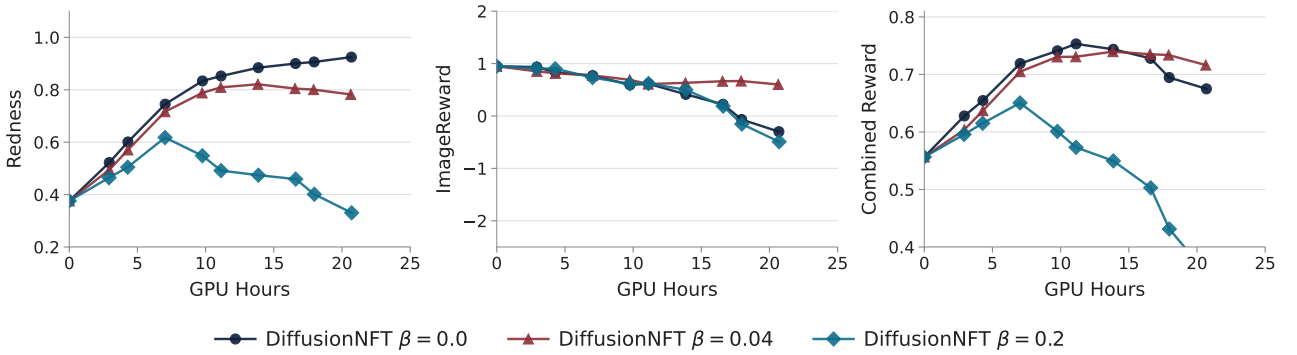


Figure C.6: Influence of different regularization on the training of DiffusionNFT. It specifically does not utilize CFG in accordance with the original implementation.

D BikeReward prompt

To assign rewards based on the presence of bicycles, we utilize the Skywork reward model, fixing its input to the following static prompt:

```
1 In this image, is there at least one real, human-rideable pedal bicycle (not
   a motorcycle, scooter, or drawing) visible, even if it is small,
   partially occluded, or cut off at the image boundary? Is the image also
   not too noisy? Answer with a single word: yes or no.
2 Answer:
```